

RTL Design Sherpa

amber — Blocking MESI Snoopy L1 Cache Micro-Architecture Specification 1.0

October 6, 2026

Table of Contents

1 Amber Mas Index.....	17
2 amber — Blocking MESI Snoopy L1 Cache — Hardware Architecture Specification.....	18
2.1 Document Organization.....	18
2.1.1 Front Matter.....	18
2.1.2 Chapter 1: Introduction.....	18
2.1.3 Chapter 2: System Overview.....	18
2.1.4 Chapter 3: Architecture.....	19
2.1.5 Chapter 4: Interfaces.....	19
2.1.6 Chapter 5: Parameters.....	19
2.1.7 Chapter 6: Integration.....	19
2.2 Figures.....	19
2.3 Related Documentation.....	20
3 Document Information.....	22
3.1 References.....	22
3.1.1 Related Documents.....	22
3.2 Terminology.....	22
3.3 Revision History.....	24
4 Architecture Overview.....	25
4.1 Introduction.....	25
4.1.1 What this book covers that the HAS does not.....	25
4.2 High-Level Block Diagram.....	26

4.3 Key Architectural Concepts.....	26
4.3.1 1. Blocking Pipeline.....	26
4.3.2 2. Dual-Port Arrays.....	27
4.3.3 3. Snoop Responder and Ordering.....	27
4.3.4 4. No Register Block.....	27
4.4 Design Decisions.....	27
4.5 Related Documentation.....	28
5 Top-Level Port List.....	29
5.1 amber — Pair-Rig Top.....	29
5.1.1 Clock and Reset.....	29
5.1.2 CPU GAXI Slave.....	29
5.1.3 AXI4 Memory Masters.....	29
5.1.4 ACE Snoop Responder.....	31
5.1.5 MonBus.....	32
5.2 amber_ace — Onyx-Rig Top.....	32
5.3 Parameters.....	33
6 Clocks and Reset.....	34
6.1 Clock Domains.....	34
6.2 Reset.....	34
7 amber_control: Blocking Pipeline FSM.....	35
7.1 Overview.....	35
7.2 State Encoding.....	35
7.2.1 FSM states inside amber_control.....	35

7.3 Pipeline Staging.....	37
7.3.1 Stage 0: Request latch.....	37
7.3.2 Stage 1: Tag lookup (CTRL_LOOKUP).....	37
7.3.3 Stage 2: Hit or miss decision.....	37
7.3.4 Stage 3: Miss handling.....	37
7.3.5 Snoop interleaving.....	37
7.4 Key Control Signals.....	38
7.5 Miss-Path Decision K-map.....	39
8 Pending-Fill Bypass Register.....	40
8.1 Purpose.....	40
8.2 Register Fields.....	40
8.3 Match Logic.....	41
8.4 Timing.....	41
8.5 Snoop vs Fill Ordering.....	41
9 amber_tag_array and amber_data_array.....	43
9.1 Overview.....	43
9.2 amber_tag_array.....	43
9.2.1 Tag + state packing.....	43
9.2.2 Port assignment.....	44
9.2.3 Addressing.....	44
9.2.4 Banking.....	44
9.3 amber_data_array.....	44
9.3.1 Organization.....	44

9.3.2 Port assignment.....	44
9.3.3 Write masking.....	45
9.3.4 Snoop data readout.....	45
9.4 Reset Behavior.....	45
10 amber_repl: Replacement Policy Engine.....	46
10.1 Overview.....	46
10.2 Interface.....	46
10.3 LRU (True LRU).....	47
10.3.1 Implementation.....	47
10.4 FIFO.....	47
10.5 RANDOM.....	48
10.6 Tree-PLRU.....	48
10.7 Policy Selection.....	48
11 amber_victim: Depth-1 Victim Buffer.....	50
11.1 Overview.....	50
11.2 Fields.....	50
11.3 Handshake with amber_control.....	50
11.4 Bypass for Snoops.....	51
11.5 Timing.....	51
12 amber_fill and amber_drain.....	52
12.1 amber_fill.....	52
12.1.1 Purpose.....	52
12.1.2 Interface.....	52

12.1.3 Burst parameters.....	52
12.1.4 Timing.....	52
12.2 amber_drain.....	53
12.2.1 Purpose.....	53
12.2.2 Interface.....	53
12.2.3 Burst parameters.....	53
12.2.4 Timing.....	53
12.2.5 On the onyx rig.....	54
13 amber_snoop_resp: Snoop Responder Adapter.....	55
13.1 Overview.....	55
13.2 External Interface.....	55
13.3 Internal Interface.....	56
13.4 Adapter State Machine.....	56
13.4.1 State transitions.....	56
13.5 CR-after-CDLAST Ordering.....	57
13.6 Snoop Type Decoding.....	57
14 amber_ace_issue: Coherent Transaction Mapping.....	59
14.1 Overview.....	59
14.2 Event-to-Transaction Mapping.....	59
14.3 Interface.....	60
14.3.1 From amber_control / fill / drain.....	60
14.3.2 To axi4ace_master_rd / axi4ace_master_wr.....	60
14.4 CleanUnique / MakeUnique Data Behavior.....	60

14.5 Timing.....	61
15 amber_cpu_frontend and amber_monlite.....	62
15.1 amber_cpu_frontend.....	62
15.1.1 Purpose.....	62
15.1.2 Request payload.....	62
15.1.3 Response payload.....	62
15.1.4 Handshake.....	62
15.1.5 Replay path.....	63
15.2 amber_monlite.....	63
15.2.1 Purpose.....	63
15.2.2 Observation points.....	63
15.2.3 Packet format.....	63
15.2.4 Drop-and-count.....	64
16 CPU GAXI Request/Response Timing.....	65
16.1 Channel Structure.....	65
16.2 Request Timing.....	65
16.3 Response Timing.....	66
16.4 Write-Allocate Fills.....	66
17 Fabric AXI4 Master Timing.....	67
17.1 Read (Fill) Timing.....	67
17.2 Write (Drain) Timing.....	67
17.3 Single Outstanding.....	68
18 Fabric ACE Master Timing.....	69

18.1 Read Transactions.....	69
18.2 Write Transactions.....	69
18.3 Timing Example: ReadShared Fill.....	69
18.4 Timing Example: WriteBack Drain.....	70
19 Snoop AC/CD/CR Timing and Ordering.....	71
19.1 External ACE Snoop Port.....	71
19.2 Transaction Sequence.....	71
19.3 CR-After-CDLAST Rule.....	72
19.4 Multiple Outstanding Snoops.....	72
19.5 AC Ready Backpressure.....	72
20 MonBus Event Emission Points.....	73
20.1 Packet Format.....	73
20.2 Emission Timing.....	73
20.3 Drop-and-Count.....	73
20.4 Backpressure Integrity.....	74
21 MonBus Event Encodings and Emission Points.....	75
21.1 Overview.....	75
21.2 Event Encoding.....	75
21.3 Payload Fields.....	76
21.4 Miss-Class Encoding.....	76
22 Drop-and-Count Behavior and monbus_tally_axil.....	78
22.1 Drop-and-Count Policy.....	78
22.2 Counter Width.....	78

22.3 monbus_tally_axil Integration.....	78
22.4 Software View.....	78
23 What the K-maps Prove.....	79
23.1 Methodology.....	79
23.2 The Three Verdicts.....	79
23.3 First-Verdict Status (2026-10-06).....	80
24 RTL Landing Diff Procedure.....	81
24.1 When RTL Lands.....	81
24.2 Diff Checklist.....	81
24.3 Regression Rule.....	81
25 gem5 FSM Cross-Check.....	83
25.1 Source.....	83
25.2 Cross-Check Mechanism.....	83
25.3 Scope.....	83
26 Elaboration Configurations.....	84
26.1 Default Config.....	84
26.2 Tiny Formal Config.....	84
26.3 Bring-Up Config.....	84
27 DV Matrix Dimensions.....	86
27.1 Pattern-B Cocotb Grids.....	86
27.1.1 Dimensions.....	86
27.1.2 Grid Coverage.....	86
27.2 Formal Targets.....	86

27.3 Golden-Model Parity.....	87
27.4 MonBus Coverage.....	87
28 amber — Product Requirements (DRAFT).....	88
28.1 1. Purpose.....	88
28.2 2. Background.....	88
28.3 3. Decisions that pick the design.....	89
28.4 4. Success criteria.....	98
28.5 5. Non-goals.....	98
28.6 6. Where the block sits.....	98
29 amber — Pre-Hardware Architecture Specification (Pre-HAS).....	99
29.1 1. Intent of this document.....	99
29.2 2. Feature sketches.....	99
29.2.1 2.1 Core datapath — the blocking bet.....	99
29.2.2 2.2 Coherence.....	100
29.2.3 2.3 Fabric side — where the two deployment rigs meet.....	101
29.2.4 2.4 Observation (D8 DECIDED — sketched, not re-decided).....	102
29.2.5 2.5 Verification & formal surface (D9 OPEN — direction proposed). .	102
29.2.6 2.6 Cost & characterisation (PRD criterion 4).....	102
29.3 3. Proposed block architecture.....	103
29.4 4. Interface sketches (shapes, not pinouts).....	104
29.5 5. Parameter sketch (illustrative — feeds the DV matrix).....	105
29.6 6. Open questions this sketch surfaces.....	106
29.7 7. What the full HAS will add.....	107

29.8 8. Cross-references.....	107
30 jet — Product Requirements (DRAFT).....	109
30.1 1. Purpose.....	109
30.2 2. Decisions jet adds to amber's.....	109
30.3 3. Success criteria (additions to amber's).....	111
30.4 4. Non-goals (for jet, beyond amber's).....	111
31 onyx — Product Requirements (DRAFT).....	112
31.1 1. Purpose.....	112
31.2 2. Background.....	112
31.3 3. Decisions that pick the design.....	113
31.4 4. Success criteria.....	121
31.5 5. Non-goals.....	121
31.6 6. Where the block sits.....	121
32 AMBA ACE — Working Interface Definition for cache-ip.....	122
32.1 1. What ACE is.....	122
32.2 2. Channel inventory.....	122
32.2.1 2.1 Coherent master → manager (front side, per master).....	122
32.2.2 2.2 Manager → cache, cache → manager (snoop side, per cache).....	123
32.3 3. Transaction types (the onyx D2 research subset in bold).....	124
32.4 4. The CRRESP decision bits.....	125
32.5 5. ACE-Lite.....	126
32.6 6. What this family implements, and what it deliberately does not.....	126
32.7 7. Sources.....	126

33	gem5 Ruby protocol specs (reference extract).....	128
34	Cache Associativity Simulator.....	129
34.1	Files.....	129
34.2	Model API.....	129
34.3	Running tests.....	130
34.4	UI features.....	130
35	Global Hard Requirements for RTL Design Sherpa.....	131
35.1	Overview.....	131
35.2	Category 1: RTL Standards.....	131
35.2.1	1.1 Reset Handling (MANDATORY - NO EXCEPTIONS).....	131
35.2.2	1.2 FPGA Synthesis Attributes (MANDATORY).....	132
35.2.3	1.3 Array Syntax Standard (MANDATORY).....	133
35.2.4	1.4 SRAM Module Standards (MANDATORY).....	133
35.2.5	1.5 Reset Convention (STANDARD).....	134
35.3	Category 2: Testbench Architecture.....	134
35.3.1	2.1 TB Location Requirements (MANDATORY).....	134
35.3.2	2.2 Three Mandatory TB Methods (MANDATORY).....	135
35.3.3	2.3 TBBase Inheritance (MANDATORY).....	135
35.3.4	2.4 Three-Layer Architecture (MANDATORY).....	136
35.3.5	2.5 Queue-Based Verification (PREFERRED).....	137
35.4	Category 3: Test Standards.....	137
35.4.1	3.1 Test Naming Convention (MANDATORY).....	137
35.4.2	3.2 Test Pattern - CocoTB + Pytest (MANDATORY for projects/components/).....	138

35.4.3 3.3 100% Success Requirement (MANDATORY).....	138
35.4.4 3.4 Simulation Timestamps in Logs (MANDATORY).....	139
35.5 Category 4: Framework Usage.....	140
35.5.1 4.1 Search Before Creating (MANDATORY).....	140
35.5.2 4.2 BFM Usage for Protocols (MANDATORY for RAPIDS).....	140
35.6 Category 5: Documentation.....	141
35.6.1 5.1 No Emojis in Technical Docs (MANDATORY).....	141
35.7 Category 6: Project-Specific Requirements.....	141
35.7.1 6.1 Commit Messages Carry No Attribution Trailer.....	141
35.7.2 6.2 STREAM Tutorial Focus.....	141
35.8 Summary by Priority.....	142
35.8.1 P0 - Critical (Enforcement Required).....	142
35.8.2 P1 - High Priority (Strong Recommendation).....	142
35.8.3 P2 - Standard Practice (Best Practices).....	142
35.8.4 P3 - Project-Specific.....	142
35.9 Compliance Checklist.....	143
35.10 Enforcement.....	143
35.11 References.....	144

List of Figures

No figures in this document.

List of Tables

Table 1: Table 0.0: Figures and their sources.....	19
Table 2: Table 0.1: Related documentation.....	20
Table 3: Related Documents and Specifications.....	22
Table 4: amber MAS Document Revision History.....	24
Table 5: Table 1.3.1: Clock domains.....	34
Table 6: Table 1.3.2: Reset signals.....	34
Table 7: Table 2.8.1: Cache events to ACE transactions.....	59
Table 8: Table 3.1.1: CPU GAXI channels.....	65
Table 9: Table 4.1.1: MonBus event encodings.....	75

List of Waveforms

No waveforms in this document.

1 Amber Mas Index

Generated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

2 amber — Blocking MESI Snoopy L1 Cache — Hardware Architecture Specification

Version: 1.0 **Date:** 2026-10-06 **Status:** full HAS ratifies the Pre-HAS sketch; PRD decisions D1, D2, D4, D5, D6, D7, D8, D9, D10 are DECIDED. D3 (memory-side shape) and D11 (array construction) remain OPEN and are recorded honestly in the chapters they shape.

Scope: The architecture of `amber` (pair-rig top) and `amber_ace` (onyx-rig top) from the CPU GAXI slave through the shared cache core to the fabric masters and ACE snoop responder. The [MAS](#) closes the micro-architecture decisions this book defers.

How this book relates to its sources. The [amber PRD](#) is the binding requirements document. This HAS carries the Pre-HAS feature sketches (F1–F18) forward, drops their “PROPOSED” hedging where the PRD has decided, and fixes the module set and interfaces as architecture. Where D3 and D11 are still open, the chapters state the current direction and the consequences; nothing is disguised as decided. The [GLOBAL_REQUIREMENTS.md](#) reset-macro, array-syntax, no-bespoke-SRAM, and filelist rules apply throughout.

2.1 Document Organization

Note: All chapters linked below for automated document generation.

2.1.1 Front Matter

- [Document Information](#)

2.1.2 Chapter 1: Introduction

- [Purpose and Scope](#)
- [Document Conventions](#)
- [Definitions and Acronyms](#)

2.1.3 Chapter 2: System Overview

- [Use Cases](#)
- [Key Features](#)
- [System Context](#)
- [Block Diagram](#)

2.1.4 Chapter 3: Architecture

- [Top Structure and the Two Rigs](#)

- [Data Flow: Hits, Misses, Fills, and Drains](#)
- [Control FSM: Blocking Pipeline and Pending-Fill Bypass](#)
- [Coherence FSM and the Snoop Responder](#)
- [Arbitration and the Victim Bypass](#)

2.1.5 Chapter 4: Interfaces

- [CPU-Side GAXI Slave](#)
- [Fabric-Side AXI4 Masters](#)
- [Fabric-Side ACE Masters and Coherent Issue](#)
- [Snoop Port: AC/CD/CR Adapter](#)
- [Observation: MonBus through amber_monlite](#)

2.1.6 Chapter 5: Parameters

- [amber_pkg and Build-Time Geometry](#)

2.1.7 Chapter 6: Integration

- [Clocking and Reset](#)
- [Verification Strategy](#)
- [Characterization](#)
- [Future Hooks](#)

2.2 Figures

Table 0.0: Figures and their sources

Figure	Source	Subject
2.1	assets/mermaid/ 01_block_diagram.mmd	amber shared core, the two rig tops, and the peer/onyx snoop boundary
3.1	assets/mermaid/ 02_miss_flow.mmd	miss-to-fill-to-replay data flow, with victim drain
3.2	assets/mermaid/ 03_snoop_flow.mmd	probe path through tag port B and the pending-fill bypass

2.3 Related Documentation

Table 0.1: Related documentation

Document	Where	Relationship
amber PRD	../PRD.md	binding requirements — all D-rows and success criteria originate here
amber Pre-HAS	./amber_prehas.md	architecture sketch; this HAS supersedes and ratifies it
amber MAS	../amber_mas/amber_mas_index.md	micro-architecture and pre-RTL signal-contract / kmap workbook
jet PRD	.././jet-mesi-l1/PRD.md	lockup-free follow-on; amber must not silently pre-decide jet's J-rows
onyx PRD	.././onyx-ace-ccu/PRD.md	onyx D7 defines the ACE-shaped snoop port; onyx D2 defines the coherent-transaction subset
AMBA ACE Interface Definition	.././References/AMBA_ACE_Interface_Definition.md	in-repo ACE contract, verified against IHI 0022H
gem5 Ruby protocols	.././References/gem5-ruby-protocols/README.md	golden-protocol source for the Python reference model and FSM oracles
cache simulator	../././././bin/apps/cache_sim/README.md	golden parity model for LRU/FIFO/RANDOM
Global Requirements	../././././GLOBAL_REQUIREMENTS.md	reset macros, array syntax, no bespoke SRAM, filelist registry

Last Updated: 2026-10-06 **Maintained By:** amber architecture

3 Document Information

This document is the Micro-Architecture Specification (MAS) for *amber*, a parameterised, blocking, snoopy MESI L1 data cache. It closes the implementation decisions that the [amber Hardware Architecture Specification \(HAS\)](#) deliberately deferred to RTL bring-up: pipeline staging, array banking, the exact control FSM state encoding, replacement-policy implementation details, and the pre-RTL combinational decision-logic contracts captured in the kmap workbook.

3.1 References

3.1.1 Related Documents

Related Documents and Specifications

Source	Title	Version
RTL Design Sherpa	amber Product Requirements Document	0.5
RTL Design Sherpa	amber Hardware Architecture Specification	1.0
RTL Design Sherpa	amber Pre-HAS sketch	1.0
ARM	AMBA AXI and ACE Protocol Specification	IHI0022H
gem5	Ruby MESI_Two_Level protocol extract	commit f5c5a6e

3.2 Terminology

ACE

AXI Coherency Extensions. The ARM protocol carrying snoop channels (AC/CD/CR) and coherent master transactions.

AC / CD / CR

ACE snoop channels: Address Coherent (AC), Coherent Data (CD), Coherent Response (CR).

CRRESP

5-bit snoop response carrying DataTransfer, PassDirty, IsShared, WasUnique, and an error bit.

DT / PD / IS / WU

CRRESP sub-fields: DataTransfer[0], PassDirty[1], IsShared[2], WasUnique[3].

FUB

Functional Unit Block. A self-contained RTL module with a defined interface and testbench.

GAXI

House generic-AXI streaming pattern (wr_valid/wr_ready/wr_data, rd_valid/rd_ready/rd_data) used for the CPU-side port.

LRU / FIFO / RANDOM / tree-PLRU

Replacement policies supported by amber_repl. LRU, FIFO, and RANDOM have cache_sim golden-model parity; tree-PLRU is the timing fallback without a sim golden model today.

MAC

Macro. An integration-level block that instantiates and connects multiple FUBs.

MESI

Cache coherence states: Modified, Exclusive, Shared, Invalid.

MonBus

128-bit packet + 64-bit side-band timestamp monitor bus. amber_monlite is a drop-and-count observer that never stalls the observed path.

Pending-fill bypass register

Single register held by amber_control while a fill is outstanding, allowing snoops to the same line to be answered with the post-fill state before the array is updated.

RACK / WACK

ACE read/write acknowledge pulses. In amber's adapters they are auto-pulsed one cycle after the last read beat / B handshake.

SDPRAM

Simple dual-port RAM. The house sdpram_core primitive is the only allowed tag/data storage.

Snoop responder

`amber_snoop_resp`, the ACE-shaped adapter that presents bus-agnostic {`address`, `type`, `response`, `data`} transactions to `amber_core`.

3.3 Revision History

amber MAS Document Revision History

Rev	Date	Author	Notes
1.0	2026-10-06	seang	Initial amber MAS: index, ch01–ch06, styles, title, and pre-RTL kmap workbook. No RTL yet; all micro-architecture proposals are contracts for the landing implementation.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 Architecture Overview

Component: amber — Blocking MESI Snoopy L1 Cache **Version:** 1.0 **Status:** Pre-RTL micro-architecture contract

4.1 Introduction

The amber [Hardware Architecture Specification \(HAS\)](#) fixes the module set, interface shapes, and parameter decisions. This Micro-Architecture Specification (MAS) closes what the HAS deferred: the cycle-level behavior of every module, the exact control FSM state encoding, how tag and state pack into the dual-port `sdpram_core`, the replacement-policy datapaths, and the handshake timing of each external interface.

Because amber has **no register block**, there is no CSR programming chapter. All runtime visibility is through the MonBus observation fabric. The MAS therefore replaces the traditional register-map chapter with a MonBus observation map.

4.1.1 What this book covers that the HAS does not

1. **Per-module micro-architecture** — how `amber_control` advances its FSM, how the pending-fill bypass register is matched and timed, how `amber_tag_array` and `amber_data_array` use `sdpram_core` port A and port B, how `amber_repl` implements each policy, and how `amber_snoop_resp` stages CR and CD.
 2. **Cycle-level interface behavior** — GAXI request/response handshake timing, AXI4/ACE master channel timing, AC/CD/CR snoop ordering including the CR-after-CDLAST rule, and MonBus event emission points.
 3. **The kmap workbook as decision-logic contract** — every important combinational decision signal is captured as a K-map with axis equations, checked relations, and a mechanically-derived minimal SOP. The workbook is generated by `gen_amber_contracts_kmaps.py`. Because RTL does not yet exist, `rtl_sop` is intentionally absent; when RTL lands the derived cover will be diffed against the RTL expression and one of the three methodology verdicts applied: identical, RTL-redundant, or RTL-differs.
-

4.2 High-Level Block Diagram

`amber_core` contains all logic shared between the two rig tops:

```

amber_core
├── amber_cpu_frontend    (GAXI slave request/response latch)
├── amber_control         (blocking pipeline FSM)
├── amber_tag_array       (dual-port tag + MESI state)
├── amber_data_array      (dual-port data store)
├── amber_repl           (replacement policy engine)
├── amber_victim          (depth-1 dirty victim buffer)
├── amber_fill           (line-fill read interface)
├── amber_drain           (dirty victim write-back interface)
├── amber_snoop_resp      (ACE snoop adapter)
└── amber_monlite        (MonBus observer)

```

The pair-rig top amber wraps `amber_core` with plain AXI4 read/write masters (`axi4_master_rd/axi4_master_wr`) toward memory and the snoop responder toward the peer cache. The onyx-rig top `amber_ace` wraps the same core with `axi4ace_master_rd/axi4ace_master_wr` masters and adds `amber_ace_issue` to map cache-side events to ACE coherent transactions.

There is no RIG parameter. The rig is selected by instantiating the appropriate top module.

4.3 Key Architectural Concepts

4.3.1 1. Blocking Pipeline

`amber_control` is the only FSM in the cache. A request flows through:

1. **Lookup** — tag/state read on port A.
2. **Hit service** — data read or write on port A, response returned.
3. **Miss resolution** — ask `amber_repl` for a victim way.
4. **Victim drain** — if dirty, stage the victim and launch `amber_drain`.
5. **Fill request** — launch `amber_fill` for a whole-line burst.
6. **Fill completion** — write data and install tag + state on port A.
7. **Replay** — re-present the original request from the front-end latch; it now hits.

Because the pipeline is blocking, no new CPU request is accepted while a miss is in flight. This single-outstanding property is what makes the pending-fill bypass tractable.

4.3.2 2. Dual-Port Arrays

`amber_tag_array` and `amber_data_array` are built from the house `sdpram_core` primitive. Port A serves CPU lookups, fill writes, and replay reads. Port B serves snoop lookups and snoop

data readout. The arrays themselves need no arbitration because CPU and snoop use separate ports; contention is resolved at the control FSM level.

4.3.3 3. Snoop Responder and Ordering

`amber_snoop_resp` is the only ACE-aware block inside `amber_core`. It presents a bus-agnostic {address, type, response, data} interface to `amber_control`. The adapter completes all CD beats for a transaction before asserting CR, matching the family convention and the cocotb-framework 1.2.0 compliance checker.

4.3.4 4. No Register Block

`amber` is parameter-only. Configuration, event counting, and debug visibility all come through MonBus. This is deliberate and keeps the CPU-side GAXI port the only software-visible surface.

4.4 Design Decisions

Decision	Source	MAS consequence
GAXI CPU-side slave	PRD D2	<code>amber_cpu_frontend</code> is a GAXI slave with a latched request and in-order response stream.
Write-back + write-allocate	PRD D5	Dirty victims are drained; fills are whole-line read-modify-write inside <code>amber_control</code> .
Plain MESI on 3-bit state	PRD D6	<code>amber_pkg</code> encodes M/E/S/I; one bit is reserved for MOESI O-state.
Replacement policies {LRU, tree-PLRU, FIFO, RANDOM}	PRD D7	<code>amber_repl</code> implements all four; LRU/FIFO/RANDOM match <code>cache_sim</code> ; tree-PLRU is the timing fallback.
ACE-shaped snoop port	PRD D4	<code>amber_snoop_resp</code> drives AC-ready, CR-valid, CD-

Decision	Source	MAS consequence
MonBus observation only	PRD D8	valid; CR follows CDLAST. amber_monlite emits event packets and drops-and-counts under backpressure.

4.5 Related Documentation

- [amber HAS](#) — architecture decisions and interface contracts.
- [amber PRD](#) — binding requirements.
- [K-map workbook](#) — pre-RTL combinational contracts.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 Top-Level Port List

This chapter lists the top-level ports of `amber` and `amber_ace`. Internal `amber_core` ports are defined in the per-block chapters of Chapter 2.

5.1 `amber` — Pair-Rig Top

5.1.1 Clock and Reset

Signal	Direction	Width	Description
<code>aclk</code>	input	1	System clock
<code>aresetn</code>	input	1	Active-low asynchronous reset, synchronous deassert

5.1.2 CPU GAXI Slave

Signal	Direction	Width	Description
<code>cpu_req_wr_valid</code>	input	1	CPU request valid
<code>cpu_req_wr_ready</code>	output	1	Cache ready to accept request
<code>cpu_req_wr_data</code>	input	CPU_REQ_W	Packed {addr, we, be, wdata}
<code>cpu_rsp_rd_valid</code>	output	1	Cache response valid
<code>cpu_rsp_rd_ready</code>	input	1	CPU ready to accept response
<code>cpu_rsp_rd_data</code>	output	CPU_RSP_W	Packed {rdata}

5.1.3 AXI4 Memory Masters

Plain AXI4 masters driven by `amber_fill` and `amber_drain`. Signal names and widths follow the house `axi4_master_rd`/`axi4_master_wr` wrappers.

Channel	Direction	Signals
AR	out	<code>m_axi_arid</code> , <code>m_axi_araddr</code> ,

Channel	Direction	Signals
		m_axi_arlen, m_axi_arsize, m_axi_arburst, m_axi_arlock, m_axi_arcache, m_axi_arprot, m_axi_arqos, m_axi_arregion, m_axi_aruser, m_axi_arvalid
R	in/out	m_axi_rid, m_axi_rdata, m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid, m_axi_rready
AW	out	m_axi_awid, m_axi_awaddr, m_axi_awlen, m_axi_awsiz, m_axi_awburst, m_axi_awlock, m_axi_awcache, m_axi_awprot, m_axi_awqos, m_axi_awregion, m_axi_awuser, m_axi_awvalid
W	out	m_axi_wdata, m_axi_wstrb, m_axi_wlast, m_axi_wuser, m_axi_wvalid
B	in/out	m_axi_bid, m_axi_bresp,

Channel	Direction	Signals
		m_axi_buser, m_axi_bvalid, m_axi_bready

5.1.4 ACE Snoop Responder

Signal	Direction	Width	Description
m_axi_acaddr	input	ADDR_WIDTH	Snoop address
m_axi_acsnoop	input	4	Snoop transaction type
m_axi_acprot	input	3	Snoop protection
m_axi_acvalid	input	1	Snoop address valid
m_axi_acready	output	1	Cache ready for snoop
m_axi_crresp	output	5	Snoop response
m_axi_crvalid	output	1	Snoop response valid
m_axi_crready	input	1	Snoop response ready
m_axi_cddata	output	DATA_WIDTH	Snoop data
m_axi_cdlast	output	1	Snoop data last beat
m_axi_cdvalid	output	1	Snoop data valid
m_axi_cdready	input	1	Snoop data ready

5.1.5 MonBus

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet

Signal	Direction	Width	Description
			valid
mon_ready	input	1	Monitor consumer ready
mon_packet	output	128	Monitor packet
mon_timestamp	output	64	Side-band timestamp

5.2 amber_ace — Onyx-Rig Top

amber_ace has the same CPU GAXI slave, snoop responder, and MonBus ports as amber. The memory-side masters are full ACE masters with ARSN00P / AWSN00P fields and auto-pulsed RACK / WACK outputs. amber_ace_issue maps cache-side events to ACE transactions; see Chapter 2.8.

Signal	Direction	Width	Description
m_axi_arsnoop	output	4	ACE read transaction type
m_axi_rack	output	1	ACE read acknowledge (auto-pulsed)
m_axi_awsnoop	output	3	ACE write transaction type
m_axi_wack	output	1	ACE write acknowledge (auto-pulsed)

All other AXI/ACE master signals mirror amber but connect to axi4ace_master_rd / axi4ace_master_wr instead of the plain AXI4 wrappers.

5.3 Parameters

All geometry, policy, and bus-width choices are elaboration parameters. See Chapter 6 and the HAS [parameters chapter](#) for the complete list. There is no runtime register block.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Clocks and Reset

6.1 Clock Domains

Table 1.3.1: Clock domains

Clock	Frequency	Usage
aclk	50–200 MHz	Primary — all amber logic, AXI4/ACE/GAXI interfaces, MonBus

amber is a single-clock IP. The GAXI slave, AXI4/ACE masters, snoop responder, and MonBus all run on aclk.

6.2 Reset

Table 1.3.2: Reset signals

Signal	Polarity	Type	Usage
aresetn	Active-low	Async assert, sync deassert	Primary reset for all amber logic and memories

aresetn resets all flops. The sdpram_core instances inside amber_tag_array and amber_data_array are not reset; their contents are invalidated by clearing valid bits or by the first access after reset, depending on the implementation choice recorded in Chapter 2.3.

All sequential logic uses the house ALWAYS_FF_RST macro with RST_ASSERTED(aresetn). This is the same pattern used in the AMBA wrappers and STREAM.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

7 amber_control: Blocking Pipeline FSM

Module: `amber_control.sv` **Location:**

`projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** Pre-RTL micro-architecture contract

7.1 Overview

`amber_control` is the cache's only state machine. It owns the blocking pipeline, the tag-array port-A schedule, the pending-fill bypass register, victim-way selection, and the handshakes to `amber_fill`, `amber_drain`, `amber_victim`, and `amber_snoop_resp`. Because the cache is blocking, there is exactly one CPU transaction in flight at any time; snoops are serviced concurrently on tag-array port B.

7.2 State Encoding

`amber_pkg` defines a 3-bit cache-line state field for M, E, S, I. One extra code is reserved for a future MOESI 0 state; the control FSM v1.0 implements only MESI.

```
typedef enum logic [2:0] {  
    STATE_I = 3'b000, // Invalid  
    STATE_S = 3'b001, // Shared  
    STATE_E = 3'b010, // Exclusive  
    STATE_M = 3'b011, // Modified  
    STATE_O = 3'b100, // Reserved for future MOESI Owned state  
    // 3'b101, 3'b110, 3'b111 reserved  
} cache_state_t;
```

The 3-bit width is intentional: it gives one reserved code for 0 and three spare codes for future transient/stable elaborations without widening the tag store.

7.2.1 FSM states inside `amber_control`

The FSM itself is one-hot encoded. Stable states are the ones that can persist between CPU requests; transient states exist only while a miss or snoop is in progress, mirroring the `gem5 Ruby MESI_Two_Level-L1cache.sm` philosophy that every in-flight condition is an explicit enumerated state.

State	Type	Meaning
CTRL_IDLE	stable	Waiting for a CPU request from <code>amber_cpu_frontend</code> .

State	Type	Meaning
CTRL_LOOKUP	transient	Tag/state lookup on port A; hit/miss decision.
CTRL_HIT_RD	transient	Read hit: fetch data from <code>amber_data_array</code> port A, return response.
CTRL_HIT_WR	transient	Write hit: update data, promote to M, return response.
CTRL_MISS_VICTIM	transient	Miss: ask <code>amber_repl</code> for victim way; read victim state.
CTRL_MISS_DRAIN	transient	Victim is dirty: stage it in <code>amber_victim</code> , launch <code>amber_drain</code> .
CTRL_MISS_FILL	transient	Launch <code>amber_fill</code> , wait for RLAST.
CTRL_FILL_WRITE	transient	Write fill data into <code>amber_data_array</code> port A; install tag + state.
CTRL_REPLAY	transient	Re-present original request from front-end latch; it now hits.
CTRL_SNOOP	transient	Service a snoop on port B (can preempt / interleave with miss states).
CTRL_ERROR	stable	Sticky fatal error; cleared only by reset.

The FSM is not a minimal two-state controller; the explicit transient states are the formal proof surface. Each state asserts exactly the control signals that are legal for that state, and illegal states default to CTRL_ERROR.

7.3 Pipeline Staging

The HAS deferred pipeline staging to the MAS. This section fixes it.

7.3.1 Stage 0: Request latch

`amber_cpu_frontend` accepts a GAXI request and holds {`addr`, `we`, `be`, `wdata`} in a register. `amber_control` sees a single `req_valid` pulse and the latched request fields. The front end does not accept a new request until `req_ready` is returned.

7.3.2 Stage 1: Tag lookup (CTRL_LOOKUP)

On the cycle after request acceptance, `amber_control` drives `tag_a_addr` and `tag_a_en`. The tag array returns {`tag`, `state`} on the next cycle. The lookup also selects the victim way from `amber_repl` combinatorially so that the victim is known on the same cycle the hit/miss decision is made.

7.3.3 Stage 2: Hit or miss decision

- **Hit:** transition to `CTRL_HIT_RD` or `CTRL_HIT_WR`. The data array is accessed in the same cycle as the state check (the set/way was already decoded from the lookup).
- **Miss:** transition to `CTRL_MISS_VICTIM`.

7.3.4 Stage 3: Miss handling

1. `CTRL_MISS_VICTIM`: read the victim tag/state from the tag array and, if dirty, copy the victim line data from `amber_data_array` port A into `amber_victim`.
2. `CTRL_MISS_DRAIN`: if the victim is M, launch `amber_drain` with the address and data from `amber_victim`. Wait for the drain-done handshake.
3. `CTRL_MISS_FILL`: launch `amber_fill` with the missing line address. The pending-fill bypass register is loaded now.
4. `CTRL_FILL_WRITE`: on `fill_done`, write the fill data into `amber_data_array` port A and install the tag + new state in `amber_tag_array` port A.
5. `CTRL_REPLAY`: re-assert `req_valid` to the hit path using the original latched request; the next cycle is `CTRL_LOOKUP` again and the line is present.

7.3.5 Snoop interleaving

`amber_control` resolves CPU-vs-snoop contention with snoop priority. A pending snoop can cause the CPU pipeline to stall for one cycle at safe boundaries (after a tag-array access completes, never mid-burst). The snoop path uses `tag_b_*` and `data_b_*` ports exclusively; it never needs port A except when a snoop triggers an invalidation or downgrade, which is applied on the next available port-A cycle.

7.4 Key Control Signals

Signal	Direction	Meaning
ctrl_tag_a_addr	to amber_tag_array	Port A address (set index).
ctrl_tag_a_en	to amber_tag_array	Port A read/write enable.
ctrl_tag_a_wdata	to amber_tag_array	Port A write data {tag, state}.
ctrl_tag_a_way	to amber_tag_array	Way select for write.
ctrl_data_a_addr	to amber_data_array	Port A address {set, way, beat}.
ctrl_data_a_en	to amber_data_array	Port A byte-enable / write enable.
ctrl_data_a_wdata	to amber_data_array	Port A write data.
ctrl_repl_req	to amber_repl	Request replacement decision.
ctrl_repl_way	from amber_repl	Selected victim way.
ctrl_victim_wr	to amber_victim	Strobe to load victim line.
ctrl_fill_start	to amber_fill	Launch a fill for this line address.
ctrl_fill_done	from amber_fill	Fill completed (RLAST accepted).
ctrl_drain_start	to amber_drain	Launch a drain from amber_victim.
ctrl_drain_done	from amber_drain	Drain completed (B handshake).
ctrl_snoop_req	from amber_snoop_resp	Snoop address + type valid.
ctrl_snoop_ready	to amber_snoop_resp	Control accepts snoop this cycle.
ctrl_snoop_crresp	to amber_snoop_resp	5-bit CRRESP result.
ctrl_snoop_cddata	to amber_snoop_resp	CD beat data.

Signal	Direction	Meaning
ctrl_snoop_cdlast	to amber_snoop_resp	CD last beat.
ctrl_req_ready	to amber_cpu_frontend	Ready for next CPU request.
ctrl_rsp_valid	to amber_cpu_frontend	Response valid.
ctrl_rsp_data	to amber_cpu_frontend	Response read data.

7.5 Miss-Path Decision K-map

The combinational decision that launches drain/fill/replay is captured in the kmap workbook:

- **Outputs:** start_drain, start_fill, replay_now.
- **Axes:** {hit, victim_dirty, pending_bypass_match}.
- **Sheet:** K-maps amber control in amber_signal_contracts.xlsx, generated by gen_amber_contracts_kmaps.py.

The sheet is marked **PROPOSED** because it is a micro-architecture proposal owned by this chapter; the RTL must match the derived cover when it lands.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 Pending-Fill Bypass Register

Module: `amber_control` sub-block **Status:** Pre-RTL micro-architecture contract

8.1 Purpose

While a fill is outstanding, a snoop may arrive for the same line. The pending-fill bypass register lets `amber_control` answer that snoop with the post-fill state and any fill beats already received, before the tag and data arrays have been updated. This is the only place in `amber` where a line can be observed in a state that has not yet been committed to the arrays, so its correctness is a SymbiYosys target.

8.2 Register Fields

Field	Width	Meaning
<code>pf_addr</code>	<code>ADDR_WIDTH - OFFSET_WIDTH</code>	Line address (tag + set index) of the fill.
<code>pf_state</code>	3	State that will be installed when the fill completes.
<code>pf_data_valid</code>	<code>FILL_BEATS</code>	Per-beat mask: <code>bit[i]</code> is 1 when fill beat <code>i</code> has been received.
<code>pf_active</code>	1	Register is valid and the fill is still outstanding.

The register is loaded in `CTRL_MISS_FILL` when `amber_fill` is launched. `pf_state` is determined by the request type:

- Read miss to shared → `STATE_S`
- Read miss to exclusive → `STATE_E`
- Write miss (whole-line allocate) → `STATE_M`

`pf_data_valid` is updated as each R beat is accepted by `amber_fill`. The snoop responder can forward fill beats whose `pf_data_valid` bit is set; beats not yet received stall the CD channel until they arrive.

8.3 Match Logic

A snoop matches the pending-fill bypass when:

```
pf_active &&  
(snoop_line_addr == pf_addr)
```

The line address is {tag, set_index}; the line offset is ignored. The kmap workbook contains a small optional bypass-match sheet, but the match is a single equality compare; the sheet is included only if it earns its place as a teaching aid.

8.4 Timing

Cycle	Event
CTRL_MISS_FILL entry	pf_active set, pf_addr and pf_state loaded, pf_data_valid cleared.
Each accepted R beat	Corresponding pf_data_valid bit set by amber_fill done-strobe.
Snoop arrives during fill	If match, CRRESP and CD beats sourced from bypass register; otherwise normal tag lookup.
CTRL_FILL_WRITE	Bypass register contents written to arrays; pf_active cleared.
CTRL_REPLAY	Original request now hits the installed line.

The critical correctness property is **state-accuracy**: the bypass register must answer with the post-fill state, never the pre-fill (Invalid) state. A snoop that would invalidate the line after the fill must see the post-fill state and then be allowed to perform its invalidation in the normal snoop path.

8.5 Snoop vs Fill Ordering

If a snoop arrives for the pending line and requires data transfer, the CD beats are sourced from whichever copy is available:

- If the beat has already been received (`pf_data_valid[i] == 1`), it is forwarded directly from the fill-path register.

- If the beat has not yet arrived, the snoop responder stalls on `cd_ready` until the beat arrives, then forwards it.

The fill still completes normally into the arrays after RLAST. The snoop does not cancel the fill; any state change caused by the snoop is applied after the fill commits.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 amber_tag_array and amber_data_array

Modules: `amber_tag_array.sv`, `amber_data_array.sv` **Location:**

`projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** RTL landed 2026-10-06; this chapter is the micro-architecture contract the RTL implements. DV: `dv/tests/fub/test_amber_{tag,data}_array.py`, green at gate/func/full across the default and tiny-formal geometries.

9.1 Overview

Both arrays are per-way inferred RAMs — the same storage idiom the house FIFOs and `sdpram_core` use internally, with the GLOBAL_REQUIREMENTS 1.2 synthesis attributes. They are **not** `sdpram_core` instances: `sdpram_core` (`rtl/amba/shared`) is a FUB/AXI burst *slave*, the wrong shape for a multi-way parallel tag lookup or single-beat random data readout. Each array has one synchronous write port (one-hot way select) and two combinational lookup ports: port A serves CPU/fill traffic and port B serves snoop traffic, matching the port assignment below. Combinational read keeps the hit/miss decision and the snoop CRRESP path inside one cycle; it maps to distributed RAM (tags) / block RAM (data) rather than a wrapped burst slave.

The one-write / two-read shape exceeds a single physical 2-port BRAM; per-way flat storage ($\text{way} * \text{SETS} + \text{set}$ for tags, $\text{way} * (\text{SETS} * \text{FILL_BEATS}) + \{\text{set}, \text{beat}\}$ for data) lets synthesis bank per way at the default geometry. That is the HAS F8 dual-port decision implemented at L1-array granularity.

9.2 amber_tag_array

9.2.1 Tag + state packing

Each way has its own `sdpram_core` instance, or the ways are banked into one instance per way depending on the synthesis attributes allowed by `GLOBAL_REQUIREMENTS.md`. The data width of each instance is:

`TAG_STATE_WIDTH = TAG_WIDTH + 3`

where `TAG_WIDTH = ADDR_WIDTH - SET_INDEX_WIDTH - LINE_OFFSET_WIDTH` and the 3 bits carry `cache_state_t`.

The memory word is packed as `{tag[TAG_WIDTH-1:0], state[2:0]}` — verified by the DV against per-location values that place nonzero patterns in both fields.

9.2.2 Port assignment

Port	Traffic	Operations
A	CPU / fill	Read tag+state for lookup; write tag+state on fill or hit-state update.
B	Snoop	Read tag+state for snoop lookup.

Port A is driven by `amber_control`. Port B is driven by `amber_control` during snoop handling.

9.2.3 Addressing

The address into the tag array is the set index:

```
tag_addr = req_addr[SET_INDEX_WIDTH + OFFSET_WIDTH - 1 : OFFSET_WIDTH]
```

Way selection is done by instantiating one `sdpram_core` per way and decoding the way index on writes. On reads, all ways return their tag+state in parallel and a comparator determines the hit way.

9.2.4 Banking

For timing closure at larger geometries, each way is a separate physical memory instance. There is no cross-way banking; a fill or snoop reads all ways in parallel and writes one. This keeps the critical hit/miss decision a simple combinational comparison across `WAYS` tag outputs.

9.3 `amber_data_array`

9.3.1 Organization

The data array stores cache-line data. A line is `LINE_BYTES*8` bits wide and is split into `FILL_BEATS = LINE_BYTES / (BUS_WIDTH/8)` beats. Each way has its own `sdpram_core` instance, or one instance per way, with:

```
DATA_MEM_DEPTH = SETS * FILL_BEATS  
DATA_MEM_WIDTH = BUS_WIDTH
```

The address is `{set_index, beat_index}`.

9.3.2 Port assignment

Port	Traffic	Operations
A	CPU / fill	Read data on hit or replay; write fill

Port	Traffic	Operations
B	Snoop	beats. Read whole line for snoop data transfer.

9.3.3 Write masking

sdpram_core is instantiated with byte-write enable support. On a CPU write hit, only the bytes selected by be are written; the rest of the line is preserved. On a fill, the entire beat is written with all byte enables asserted. On a write-allocate miss, the fill data is merged with the CPU write data before installation.

9.3.4 Snoop data readout

A snoop that requires data transfer reads the matching way from port B beat-by-beat. `amber_control` drives `b_addr = {snoop_set, beat}`, increments the beat counter each cycle until the line is complete, and supplies the hit way on `b_way` (from the tag compare on port B). The data is forwarded to `amber_snoop_resp` for CD transmission.

9.4 Reset Behavior

The arrays carry **no reset port** (GLOBAL_REQUIREMENTS 1.4 — SRAM contents are not reset, and real BRAM has no reset pin). After `aresetn` deassertion, `amber_control` treats all ways as Invalid until the init walk completes; option 1 from the pre-RTL review is the contract: a hardware sequence walks all sets and writes `STATE_I` to every way (the DV TBs perform the same walk so unwritten locations cannot leak X's). The formal proofs assume all lines are Invalid after reset.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

10 amber_repl: Replacement Policy Engine

Module: `amber_repl.sv` **Location:**

`projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** RTL landed 2026-10-06; this chapter is the micro-architecture contract the RTL implements. DV: `dv/tests/fub/test_amber_repl.py`, green at gate/func/full across all four policies x {128/4, 16/2, 64/8} geometries. The TB carries an independent Python golden model per policy (tree-PLRU included — it still has no cache_sim golden; the TB model is the check).

10.1 Overview

`amber_repl` selects the victim way on a cache miss. The policy is an elaboration parameter `REPL_POLICY` with supported values `lru`, `tree_plru`, `fifo`, and `random`. LRU is the default; tree-PLRU is the timing-tight fallback.

In the RTL the parameter is declared `int` holding the `amber_repl_t` encoding from `amber_pkg` (house precedent: `fifo_sync MEM_STYLE` is typed, but a typed enum parameter receives -G overrides as raw 32-bit constants and verilator `WIDTHTRUNCs` values 2/3 — the `int` declaration keeps test-time overrides width-clean; the generate branches compare against `int'(AMBER_REPL_*)` casts).

10.2 Interface

Signal	Direction	Width	Meaning
<code>repl_req</code>	input	1	Request a victim way this cycle.
<code>repl_set</code>	input	<code>SET_INDEX_WIDTH</code>	Set index for the access.
<code>repl_hit_way</code>	input	<code>WAY_WIDTH</code>	Way that hit (for update).
<code>repl_hit</code>	input	1	A hit occurred; update policy state.
<code>repl_victim_way</code>	output	<code>WAY_WIDTH</code>	Selected victim way.
<code>repl_update</code>	input	1	A miss install occurred; update

Signal	Direction	Width	Meaning
			policy state for repl_set.

repl_hit and repl_update are mutually exclusive in the blocking pipeline: a cycle is either a hit service or a miss install, never both.

10.3 LRU (True LRU)

True LRU maintains a recency stack per set. For *WAYS* ways, each set stores $WAYS * (WAYS - 1) / 2$ ordering bits. On every hit or install, the accessed way is moved to the most-recent position and the other ways are shifted down. The victim is the least-recent way.

This matches `cache_sim` exactly and is the golden reference for policy parity.

10.3.1 Implementation

Use a per-set array of *WAYS* rank values. The rank value is 0 for most-recent, *WAYS* - 1 for least-recent. On access:

```

for (int w = 0; w < WAYS; w++) begin
    if (w == accessed_way)
        rank[set][w] = 0;
    else if (rank[set][w] < old_rank_of_accessed_way)
        rank[set][w] += 1;
end

```

The victim way is the one with `rank[set][w] == WAYS - 1`.

10.4 FIFO

Each set maintains a FIFO queue of way indices, stored as a ring with a head pointer. On a miss, the head of the queue is the victim; on an install, the ring slot at the head is overwritten with the installed way and the head advances — the popped victim slot becomes the tail, so one write plus one head bump is the whole update. On a hit, the queue is unchanged (FIFO does not recency-stack); the TB golden model encodes the same rule (`UPDATES_ON_HIT = False`).

This matches `cache_sim` exactly, under the invariant that the installed way equals the victim way — which `amber_control` guarantees because fills install into the victim way. Installing into any other way would require a mid-queue removal this ring does not perform; that is a contract, not an accident.

This matches `cache_sim` exactly.

10.5 RANDOM

Each set contains a small LFSR. On a replacement request, the LFSR value is truncated to `WAY_WIDTH` bits to select the victim way. The LFSR advances every replacement request. The seed is parameterizable for repeatable tests.

This matches `cache_sim` exactly when the same LFSR polynomial and seed are used.

10.6 Tree-PLRU

Tree-PLRU is carried as the timing-tight fallback per PRD D7. It uses `WAYS - 1` bits per set arranged as a binary tree. On each access, the tree bits point away from the accessed way. On a replacement request, the tree bits are followed to find the victim.

Tree-PLRU does **not** have a `cache_sim` golden model today. It is cross-checked by RTL self-checks and is recorded as a future `cache_sim` extension. This is an honest caveat, not a hidden assumption.

10.7 Policy Selection

The policy is selected at elaboration time; the RTL declares the parameter `int` (values are the `amber_repl_t` encodings) and generates exactly one branch:

```
parameter int REPL_POLICY = int'(AMBER_REPL_LRU)    // values:
amber_repl_t

generate
  if (REPL_POLICY == int'(AMBER_REPL_LRU)) begin : gen_lru
    // true LRU rank array
  end else if (REPL_POLICY == int'(AMBER_REPL_FIFO)) begin :
gen_fifo
  // FIFO ring array
  end else if (REPL_POLICY == int'(AMBER_REPL_RANDOM)) begin :
gen_random
  // LFSR per set
  end else if (REPL_POLICY == int'(AMBER_REPL_TREE_PLRU)) begin :
gen_tree_plru
  // binary tree PLRU
```


**end
endgenerate**

Only one policy is instantiated per build. This keeps area predictable and matches the cache_sim per-policy parity model.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 amber_victim: Depth-1 Victim Buffer

Module: `amber_victim.sv` **Location:**

`projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** Pre-RTL micro-architecture contract

11.1 Overview

`amber_victim` is a depth-1 buffer that holds one dirty cache line while its write-back (drain) is outstanding. The depth-1 size matches the single-outstanding-miss model: there can be only one dirty victim at a time because the pipeline blocks on the miss until it resolves.

11.2 Fields

Field	Width	Meaning
<code>victim_addr</code>	<code>ADDR_WIDTH</code>	Full line address (for AW).
<code>victim_data</code>	<code>LINE_BYTES*8</code>	Dirty line data.
<code>victim_valid</code>	1	Buffer is occupied.

11.3 Handshake with `amber_control`

Signal	Direction	Meaning
<code>victim_load</code>	input from <code>amber_control</code>	Load the buffer from <code>amber_data_array</code> port A this cycle.
<code>victim_addr_in</code>	input from <code>amber_control</code>	Line address to load.
<code>victim_data_in</code>	input from <code>amber_control</code>	Line data to load.
<code>victim_busy</code>	output to <code>amber_control</code>	Buffer already occupied.
<code>victim_empty</code>	output to <code>amber_control</code>	Buffer is empty.

`victim_load` asserts for one cycle in `CTRL_MISS_VICTIM` when the victim is dirty. The buffer is busy until `amber_drain` reports done.

11.4 Bypass for Snoops

If a snoop arrives for the victim line while the drain is still outstanding, `amber_victim` can supply the data without re-reading `amber_data_array`. `amber_control` checks:

```
victim_valid && (snoop_line_addr == victim_addr)
```

If matched, the snoop responder sources CD beats from `victim_data` instead of from `data_array` port B. This is the victim bypass.

11.5 Timing

Cycle	Event
CTRL_MISS_VICTIM, victim is M	<code>victim_load</code> asserted; <code>victim_valid</code> sets next cycle.
CTRL_MISS_DRAIN	<code>amber_drain</code> consumes <code>victim_addr</code> and <code>victim_data</code> ; begins AW/W burst.
Drain done (<code>ctrl_drain_done</code>)	<code>victim_valid</code> cleared.

The buffer is never loaded while valid; the single-outstanding property guarantees this cannot happen.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 amber_fill and amber_drain

Modules: `amber_fill.sv`, `amber_drain.sv` **Location:** `projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** Pre-RTL micro-architecture contract

12.1 amber_fill

12.1.1 Purpose

`amber_fill` issues whole-line read bursts to the memory-side master and forwards received beats to `amber_data_array` port A and to the pending-fill bypass register.

12.1.2 Interface

Signal	Direction	Meaning
<code>fill_start</code>	input from <code>amber_control</code>	Launch a fill for <code>fill_addr</code> .
<code>fill_addr</code>	input from <code>amber_control</code>	Line-aligned address.
<code>fill_done</code>	output to <code>amber_control</code>	All beats received (RLAST accepted).
<code>fill_beat_valid</code>	output to <code>amber_control</code> / data array	A fill beat is valid this cycle.
<code>fill_beat_data</code>	output	Fill beat data.
<code>fill_beat_idx</code>	output	Beat index within line.
<code>fill_last</code>	output	Last beat of burst.

12.1.3 Burst parameters

- `ARLEN` = `FILL_BEATS` - 1
- `ARSIZE` = $\log_2(\text{BUS_WIDTH}/8)$
- `ARBURST` = `INCR`
- `ARADDR` line-aligned

12.1.4 Timing

1. `fill_start` asserted in `CTRL_MISS_FILL`.
2. `amber_fill` asserts `fub_axi_arvalid` to the memory-side read master.
3. As R beats arrive, `fill_beat_valid` pulses; `fill_beat_idx` increments.

4. Each beat is written to `amber_data_array` port A and the `pf_data_valid` mask is updated.
 5. On RLAST acceptance, `fill_done` pulses for one cycle.
-

12.2 `amber_drain`

12.2.1 Purpose

`amber_drain` takes a dirty line from `amber_victim` and issues a whole-line write-back burst to the memory-side write master.

12.2.2 Interface

Signal	Direction	Meaning
<code>drain_start</code>	input from <code>amber_control</code>	Launch drain from <code>amber_victim</code> .
<code>drain_done</code>	output to <code>amber_control</code>	B response received.
<code>drain_beat_ready</code>	input from memory master	W-channel ready.
<code>drain_beat_valid</code>	output	W-channel valid.
<code>drain_beat_data</code>	output	W-channel data.
<code>drain_beat_strb</code>	output	W-channel strobe (all-1s for whole-line).
<code>drain_beat_last</code>	output	W-channel last.

12.2.3 Burst parameters

- `AWLEN` = `FILL_BEATS` - 1
- `AWSIZE` = $\log_2(\text{BUS_WIDTH}/8)$
- `AWBURST` = `INCR`
- `AWADDR` = `victim_addr`
- `WSTRB` = all-1s for every beat

12.2.4 Timing

1. `drain_start` asserted in `CTRL_MISS_DRAIN` when `amber_victim` is valid.
2. `amber_drain` asserts `fub_axi_awvalid` and begins driving W beats as the master accepts them.
3. After WLAST is accepted and the B response arrives, `drain_done` pulses.

12.2.5 On the onyx rig

For `amber_ace`, dirty evictions are issued as `WriteBack` transactions. `amber_ace_issue` supplies `AWSN00P` and the `AW` address; `amber_drain` supplies the `W` beats. Clean evictions are `Evict` transactions with no data.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 amber_snoop_resp: Snoop Responder Adapter

Module: `amber_snoop_resp.sv` **Location:** `projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** RTL landed 2026-10-06; this chapter is the contract the RTL implements. Wraps the house `axi4ace_snoop_slave` transport (which exists in `rtl/amba/ace` — confirmed). DV: `dv/tests/fub/test_amber_snoop_resp.py`, green at gate/func/full on both geometries; the FULL level is a 3000-snoop randomized soak over an evolving line-state model with per-transaction ACE compliance checks, not a token rerun.

13.1 Overview

`amber_snoop_resp` is the ACE-shaped snoop responder. It wraps the house `axi4ace_snoop_slave` module and adds the small state machine that sequences CR and CD for each snoop transaction. Inside `amber_core` the snoop is bus-agnostic; only `amber_snoop_resp` speaks ACE.

13.2 External Interface

The external side is the standard `axi4ace_snoop_slave` port:

Signal	Direction	Width	Purpose
<code>m_axi_acaddr</code>	input	<code>ADDR_WIDTH</code>	Snoop address
<code>m_axi_acsnoop</code>	input	4	Snoop type
<code>m_axi_acprot</code>	input	3	Snoop protection
<code>m_axi_acvalid</code>	input	1	AC valid
<code>m_axi_acready</code>	output	1	AC ready
<code>m_axi_crresp</code>	output	5	CR response
<code>m_axi_crvalid</code>	output	1	CR valid
<code>m_axi_crready</code>	input	1	CR ready
<code>m_axi_cddata</code>	output	<code>DATA_WIDTH</code>	CD data
<code>m_axi_cdlast</code>	output	1	CD last
<code>m_axi_cdvalid</code>	output	1	CD valid
<code>m_axi_cdready</code>	input	1	CD ready

13.3 Internal Interface

The core-facing side is a bus-agnostic handshake:

Signal	Direction	Meaning
ctrl_snoop_req	output to amber_control	Valid snoop address + type.
ctrl_snoop_addr	output	Snoop line address.
ctrl_snoop_type	output	Encoded snoop type.
ctrl_snoop_ready	input from amber_control	Control accepts snoop.
ctrl_crresp	input from amber_control	5-bit response.
ctrl_cddata	input from amber_control	CD beat data.
ctrl_cdlast	input from amber_control	CD last beat.
ctrl_cdvalid	input from amber_control	CD beat valid.
ctrl_cdready	output to amber_control	Adapter ready for CD beat.

13.4 Adapter State Machine

State	Meaning
SR_IDLE	Waiting for AC handshake.
SR_LOOKUP	Snoop accepted; waiting for amber_control CRRESP.
SR_DATA	Driving CD beats.
SR_RESP	Driving CR after CDLAST.

13.4.1 State transitions

1. SR_IDLE → SR_LOOKUP: AC handshake completes (`m_axi_acvalid` && `m_axi_acready`).
2. SR_LOOKUP → SR_DATA: `amber_control` returns `ctrl_crresp` and `CRRESP.DataTransfer` is 1.
3. SR_LOOKUP → SR_RESP: `amber_control` returns `ctrl_crresp` and `CRRESP.DataTransfer` is 0.

4. SR_DATA → SR_RESP: Last CD beat accepted (m_axi_cdvalid && m_axi_cdready && m_axi_cdlast).
 5. SR_RESP → SR_IDLE: CR handshake completes (m_axi_crvalid && m_axi_crready).
-

13.5 CR-after-CDLAST Ordering

amber strengthens the protocol by completing all CD beats before asserting CR. This matches:

- IHI0022: the snoop response is required only after the final CD beat.
- The cocotb-framework 1.2.0 ACE compliance checker (check_cr_order / check_cd_order).
- The family convention recorded with onyx D4.

The CR channel is held low while the adapter is in SR_DATA. This makes the gather logic simple: one closed response at a time.

13.6 Snoop Type Decoding

m_axi_acsnoop carries the six IHI0022 snoop encodings used by amber:

Encoding	Type
ReadShared	Shared read snoop
ReadOnce	Single read snoop
ReadUnique	Exclusive read snoop
CleanShared	Clean shared snoop
CleanInvalid	Clean invalidate snoop
MakeInvalid	Invalidate snoop

CleanUnique and MakeUnique are transactions a master issues, not snoop encodings, so they do not appear on the AC channel. The CRRESP bits for each {state, snoop} pair are the authority of the HAS Table 3.0 and are captured as K-maps in the workbook.

Last Updated: 2026-10-06

14 amber_ace_issue: Coherent Transaction Mapping

Module: `amber_ace_issue.sv` **Location:** `projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** Pre-RTL micro-architecture contract

14.1 Overview

`amber_ace_issue` maps cache-side events to the onyx D2 coherent-transaction subset. It lives only in the `amber_ace` rig; the pair-rig `amber` top does not instantiate it. The module sits between `amber_control` / `amber_fill` / `amber_drain` and the `axi4ace_master_rd` / `axi4ace_master_wr` wrappers.

14.2 Event-to-Transaction Mapping

Table 2.8.1: Cache events to ACE transactions

Cache event	ACE transaction	Snoop field	Notes
Read miss, shared intent	ReadShared	<code>fub_axi_arsnoop[3:0]</code>	Line will be installed in S.
Read miss, exclusive intent	ReadUnique	<code>fub_axi_arsnoop[3:0]</code>	Line will be installed in E.
Write to Shared line	CleanUnique	<code>fub_axi_awsnoop[2:0]</code>	Upgrades S → M without data transfer.
Write to Invalid line (whole line)	MakeUnique	<code>fub_axi_awsnoop[2:0]</code>	Invalidates other copies; line installed in M.
Dirty eviction	WriteBack	<code>fub_axi_awsnoop[2:0]</code>	Whole-line write-back with data.
Clean eviction	Evict	<code>fub_axi_awsnoop[2:0]</code>	No data; notifies interconnect.

14.3 Interface

14.3.1 From `amber_control` / fill / drain

Signal	Direction	Meaning
<code>ace_rd_req</code>	input	Read transaction request.
<code>ace_rd_type</code>	input	Encoded event type (shared read / unique read).
<code>ace_rd_addr</code>	input	Line-aligned address.
<code>ace_rd_len</code>	input	Burst length.
<code>ace_wr_req</code>	input	Write transaction request.
<code>ace_wr_type</code>	input	Encoded event type (CleanUnique / MakeUnique / WriteBack / Evict).
<code>ace_wr_addr</code>	input	Line-aligned address.
<code>ace_wr_len</code>	input	Burst length.

14.3.2 To `axi4ace_master_rd` / `axi4ace_master_wr`

`amber_ace_issue` drives the `fub_axi_*` ports of the ACE master wrappers. It supplies `arsnoop/awsnop` and the standard AXI address/data fields. The wrappers auto-pulse `RACK/WACK`.

14.4 CleanUnique / MakeUnique Data Behavior

- CleanUnique is an AW-only transaction (no W data). It tells the interconnect to invalidate other copies so the local write can proceed. `amber_control` performs the actual data write into `amber_data_array` and promotes the line to M.
- MakeUnique is also an AW-only transaction for the full-line case. It invalidates other copies and grants exclusive ownership; the local line is installed or promoted to M.
- WriteBack is an AW+W burst with the dirty victim data.

- Evict is an AW-only transaction with no data.
-

14.5 Timing

`amber_ace_issue` is combinational for address/snoop mapping. It does not add pipeline stages; the timing path is from `amber_control` through this module into the `axi4ace_master_*` skid buffers.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 amber_cpu_frontend and amber_monlite

Modules: `amber_cpu_frontend.sv`, `amber_monlite.sv` **Location:** `projects/components/cache-ip/amber-mesi-l1/rtl/fub/` **Status:** Pre-RTL micro-architecture contract

15.1 amber_cpu_frontend

15.1.1 Purpose

`amber_cpu_frontend` is the GAXI slave that accepts CPU requests and returns responses. Because the cache is blocking, it latches one request at a time and does not accept a new request until `amber_control` returns `req_ready`.

15.1.2 Request payload

$\text{CPU_REQ_W} = \text{ADDR_WIDTH} + 1 + \text{BUS_WIDTH}/8 + \text{BUS_WIDTH}$

Packed as `{addr[ADDR_WIDTH-1:0], we, be[BUS_WIDTH/8-1:0], wdata[BUS_WIDTH-1:0]}`.

15.1.3 Response payload

$\text{CPU_RSP_W} = \text{BUS_WIDTH}$

Packed as `{rdata[BUS_WIDTH-1:0]}`.

15.1.4 Handshake

Signal	Direction	Meaning
<code>cpu_req_wr_valid</code>	input	CPU has a request.
<code>cpu_req_wr_ready</code>	output	Frontend accepts request this cycle.
<code>cpu_req_wr_data</code>	input	Request payload.
<code>cpu_rsp_rd_valid</code>	output	Response ready.
<code>cpu_rsp_rd_ready</code>	input	CPU accepts response.
<code>cpu_rsp_rd_data</code>	output	Response payload.

`cpu_req_wr_ready` is asserted only when `amber_control` is in `CTRL_IDLE` and no snoop priority stall is active. Once a request is accepted, the front end latches it and holds it until `amber_control` returns the response.

15.1.5 Replay path

On a miss, `amber_control` re-drives the original latched request into its own hit logic in `CTRL_REPLAY`. The front end does not see a new GAXI transaction; the response is generated from the same latched request.

15.2 `amber_monlite`

15.2.1 Purpose

`amber_monlite` is the drop-and-count MonBus observer. It taps the internal signals of `amber_core` and emits 128-bit MonBus event packets. It never stalls the observed path; under sustained backpressure it drops packets and counts the loss.

15.2.2 Observation points

Event	Emit point	Payload highlights
hit	<code>CTRL_HIT_RD</code> / <code>CTRL_HIT_WR</code>	set, way, old/new state
miss	<code>CTRL_MISS_VICTIM</code>	set, way, miss class
snoop	snoop accepted	snoop type, hit/miss, response class
eviction	<code>CTRL_MISS_VICTIM</code> dirty victim	clean/dirty, set, way
MESI transition	state update	old state, new state, cause
fill start	<code>CTRL_MISS_FILL</code>	address, direction
fill end	<code>CTRL_FILL_WRITE</code>	address
drain start	<code>CTRL_MISS_DRAIN</code>	address
drain end	drain done	address

15.2.3 Packet format

Standard MonBus 128-bit packet with house UNIT/AGENT ids. The exact field layout follows the shared `monitor_common_pkg` spec; the MAS does not invent a new layout.

15.2.4 Drop-and-count

If `mon_ready` is low when a packet is ready, the packet is dropped and a saturating drop counter increments. The drop count is reported as an `EVENT_DROPPED` packet when the bus next has room, exactly like the `STREAM` monitor-lite behavior.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 CPU GAXI Request/Response Timing

16.1 Channel Structure

The CPU-side port is two house-GAXI streams:

Table 3.1.1: CPU GAXI channels

Signal	Direction	Width	Payload
cpu_req_wr_valid	in	1	request valid
cpu_req_wr_ready	out	1	request accepted
cpu_req_wr_data	in	CPU_REQ_W	packed {addr, we, be, wdata}
cpu_rsp_rd_valid	out	1	response valid
cpu_rsp_rd_ready	in	1	response accepted
cpu_rsp_rd_data	out	CPU_RSP_W	packed {rdata}

16.2 Request Timing

Because the cache is blocking, `cpu_req_wr_ready` is asserted only when `amber_control` is in `CTRL_IDLE` and no higher-priority snoop stall is active.

```
Cycle : 0  1  2  3  4  5
req_v  : 1  0  0  0  0  0
req_r  : 1  0  0  0  0  0  <- handshake at cycle 0
state  : L0 L1 HR HR RSP RSP
```

- Cycle 0: request accepted; `amber_cpu_frontend` latches it.
- Cycle 1: `CTRL_LOOKUP` drives tag-array port A.
- Cycle 2–3: hit service (`CTRL_HIT_RD` or `CTRL_HIT_WR`).
- Cycle 4–5: response valid until accepted.

On a miss, `cpu_req_wr_ready` stays low from cycle 1 through the entire miss sequence. The front end does not accept a new request until the replay completes.

16.3 Response Timing

`cpu_rsp_rd_valid` is asserted when `amber_control` enters the response state. The data is held stable until `cpu_rsp_rd_ready` is high. Because the response is single-beat and the front end blocks, backpressure is expected to be rare, but the handshake is still honored.

16.4 Write-Allocate Fills

A write miss triggers a fill, a merge of the CPU write data with the fill data, and then a replay of the write as a hit. The GAXI slave sees only the original single-beat write request and the eventual single-beat write response; the read-modify-write sequence is internal to `amber_control`.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

17 Fabric AXI4 Master Timing

The amber pair-rig top uses the house `axi4_master_rd` and `axi4_master_wr` wrappers for plain AXI4 memory access. This chapter documents the cycle-level handoff between `amber_fill` / `amber_drain` and those wrappers.

17.1 Read (Fill) Timing

`amber_fill` drives the `fub_axi_*` side of `axi4_master_rd`:

Cycle	:	0	1	2	3	4	5	6	
AR_v	:	1	0	0	0	0	0	0	
AR_r	:	1	0	0	0	0	0	0	<- AR handshake
R_v	:	0	0	0	1	1	1	0	
R_r	:	0	0	0	1	1	1	0	<- R beats accepted
last	:	0	0	0	0	0	1	0	

- $ARLEN = FILL_BEATS - 1$, $ARSIZE = \log_2(BUS_WIDTH/8)$, $ARBURST = INCR$.
- `RREADY` is asserted as long as `amber_data_array` port A can accept the beat.
- Each accepted R beat is written to `amber_data_array` and the pending-fill bypass register is updated.
- `fill_done` pulses one cycle after `RLAST` is accepted.

17.2 Write (Drain) Timing

`amber_drain` drives the `fub_axi_*` side of `axi4_master_wr`:

Cycle	:	0	1	2	3	4	5	6	
AW_v	:	1	0	0	0	0	0	0	
AW_r	:	1	0	0	0	0	0	0	<- AW handshake
W_v	:	0	1	1	1	0	0	0	
W_r	:	0	1	1	1	0	0	0	<- W beats accepted
last	:	0	0	0	1	0	0	0	
B_v	:	0	0	0	0	0	1	0	
B_r	:	0	0	0	0	0	1	0	<- B handshake

- $AWLEN = FILL_BEATS - 1$, $AWSIZE = \log_2(BUS_WIDTH/8)$, $AWBURST = INCR$.
- `WSTRB` is all-1s for every beat.
- `drain_done` pulses one cycle after the B handshake.

17.3 Single Outstanding

There is at most one AR transaction and one AW transaction in flight at any time because the blocking pipeline does not launch a new miss until the previous one completes. No reorder buffer is needed inside `amber_fill` or `amber_drain`.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

18 Fabric ACE Master Timing

The `amber_ace` onyx-rig top uses the house `axi4ace_master_rd` and `axi4ace_master_wr` wrappers. The timing is identical to the AXI4 case except for the ARSNOOP/AWSNOOP fields and the auto-pulsed RACK/WACK outputs.

18.1 Read Transactions

Cache event	ACE transaction	fub_axi_arsnoop
read miss, shared intent	ReadShared	ReadShared encoding
read miss, exclusive intent	ReadUnique	ReadUnique encoding

RACK is auto-pulsed one cycle after RLAST is accepted. `amber_ace_issue` supplies `fub_axi_arsnoop`; `amber_fill` supplies address, length, size, and burst.

18.2 Write Transactions

Cache event	ACE transaction	fub_axi_awsnoop
write to Shared line	CleanUnique	CleanUnique encoding
write to Invalid line (whole line)	MakeUnique	MakeUnique encoding
dirty eviction	WriteBack	WriteBack encoding
clean eviction	Evict	Evict encoding

WACK is auto-pulsed one cycle after the B handshake. CleanUnique, MakeUnique, and Evict are AW-only transactions with no W data. WriteBack is an AW+W burst with the dirty victim data.

18.3 Timing Example: ReadShared Fill

Cycle :	0	1	2	3	4	5	6	
AR_v :	1	0	0	0	0	0	0	
AR_r :	1	0	0	0	0	0	0	<- AR handshake,
ARSNOOP=ReadShared								
R_v :	0	0	0	1	1	1	0	
R_r :	0	0	0	1	1	1	0	
last :	0	0	0	0	0	1	0	
rack :	0	0	0	0	0	0	1	<- auto-pulsed

18.4 Timing Example: WriteBack Drain

Cycle	:	0	1	2	3	4	5	6	
AW_v	:	1	0	0	0	0	0	0	
AW_r	:	1	0	0	0	0	0	0	<- AW handshake,
AWSN00P=WriteBack									
W_v	:	0	1	1	1	0	0	0	
W_r	:	0	1	1	1	0	0	0	
last	:	0	0	0	1	0	0	0	
B_v	:	0	0	0	0	0	1	0	
B_r	:	0	0	0	0	0	1	0	
wack	:	0	0	0	0	0	0	1	<- auto-pulsed

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 Snoop AC/CD/CR Timing and Ordering

19.1 External ACE Snoop Port

Signal	Direction	Purpose
m_axi_acvalid/ m_axi_acready	in / out	AC handshake
m_axi_acaddr	in	snoop line address
m_axi_acsnoop	in	snoop type
m_axi_cdvalid/ m_axi_cdready	out / in	CD handshake
m_axi_cddata	out	snoop data beat
m_axi_cdlast	out	last CD beat
m_axi_crvalid/ m_axi_crready	out / in	CR handshake
m_axi_crresp	out	5-bit response

19.2 Transaction Sequence

Cycle	: 0	1	2	3	4	5	6	7	
AC_v	: 1	0	0	0	0	0	0	0	
AC_r	: 1	0	0	0	0	0	0	0	<- AC handshake
lookup:		L0	L1						
CD_v	: 0	0	0	1	1	1	0	0	
CD_r	: 0	0	0	1	1	1	0	0	
last	: 0	0	0	0	0	1	0	0	
CR_v	: 0	0	0	0	0	0	1	0	
CR_r	: 0	0	0	0	0	0	1	0	<- CR after CDLAST

- Cycle 0: AC handshake. The snoop address and type are captured.
- Cycles 1–2: `amber_control` performs tag lookup on port B. If the line is M/E and the snoop requires data, it reads the data array port B.
- Cycles 3–5: CD beats are driven and accepted.
- Cycle 6: CR is asserted after CDLAST has been accepted.
- Cycle 7: CR handshake completes; the adapter returns to idle.

19.3 CR-After-CDLAST Rule

`amber_snoop_resp` holds `m_axi_crvalid` low until the last CD beat has been accepted. This is stronger than the minimum IHI0022 requirement but matches the family convention and the cocotb-framework 1.2.0 compliance checker. It also makes the gather logic simple: the manager sees one closed response at a time.

19.4 Multiple Outstanding Snoops

The `axi4ace_snoop_slave` wrapper buffers up to `SKID_DEPTH_AC` snoops. However, `amber_control` processes them one at a time because it is a single FSM. Therefore the effective outstanding snoop count is 1 from the core's perspective; the adapter FIFO absorbs bursts of AC arrivals.

19.5 AC Ready Backpressure

`m_axi_acready` is the skid-buffer `wr_ready` from `axi4ace_snoop_slave`. It can be low when the AC skid is full. The manager must respect this; amber never drops a snoop address because the adapter buffers it.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

20 MonBus Event Emission Points

20.1 Packet Format

`amber_monlite` emits standard 128-bit MonBus packets plus a 64-bit side-band timestamp. The packet layout is the house `monitor_common_pkg` spec; the timestamp is sampled at the same cycle as the packet and held stable while `mon_valid` is high.

20.2 Emission Timing

Events are emitted as one-cycle pulses. The event class and payload are captured combinatorially from the control FSM state and registered at the emission point to avoid glitches.

Event class	Emit point	Payload
hit	CTRL_HIT_RD / CTRL_HIT_WR	set, way, state before/after
miss	CTRL_MISS_VICTIM	set, way, miss class
snoop	AC handshake accepted	snoop type, hit/miss flag
eviction	CTRL_MISS_DRAIN start	clean/dirty, set, way
MESI transition	tag-array state write	old state, new state, cause code
fill_start	CTRL_MISS_FILL	line address
fill_end	CTRL_FILL_WRITE	line address
drain_start	CTRL_MISS_DRAIN	line address
drain_end	drain_done	line address

20.3 Drop-and-Count

If `mon_ready` is low when `mon_valid` would assert, the packet is dropped and a saturating drop counter increments. The counter is reported as an `EVENT_DROPPED` packet when MonBus next has room. This is identical to the STREAM monitor-lite behavior and never stalls the observed path.

20.4 Backpressure Integrity

`amber_monlite` is a pure observer. Its FIFO state never gates the CPU GAXI port, the AXI4/ACE masters, or the snoop responder. This is the measurement-integrity rationale that chose `*_monlite` over the heavyweight `_mon` wrappers.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 MonBus Event Encodings and Emission Points

21.1 Overview

Because amber has no register block, the MonBus observation map replaces the traditional register-map chapter. Every software-visible event is emitted as a MonBus packet by `amber_monlite`.

21.2 Event Encoding

Table 4.1.1: MonBus event encodings

Event code	Name	Emitting module	Trigger
AMBER_EV_HIT	hit	amber_monlite	CTRL_HIT_RD or CTRL_HIT_WR
AMBER_EV_MISS	miss	amber_monlite	CTRL_MISS_VI CTIM
AMBER_EV_SNOOP	snoop	amber_monlite	AC handshake accepted
AMBER_EV_EVICT	eviction	amber_monlite	dirty victim staged to amber_victim
AMBER_EV_TRANSITION	MESI transi- tion	amber_monlite	tag-array state write
AMBER_EV_FILL_START	fill start	amber_monlite	CTRL_MISS_FI LL
AMBER_EV_FILL_END	fill end	amber_monlite	CTRL_FILL_WR ITE
AMBER_EV_DRAIN_START	drain start	amber_monlite	CTRL_MISS_DR AIN
AMBER_EV_DRAIN_END	drain end	amber_monlite	drain_done
AMBER_EV_DROPPED	dropped packet	amber_monlite	packet dropped due to backpressure

The exact numeric codes are assigned in `amber_pkg` and match the `house_monitor_common_pkg` allocation rules.

21.3 Payload Fields

All packets carry:

- `agent_id` — house `MON_AGENT_ID` for amber.
- `unit_id` — sub-unit within amber (control, snoop, etc.).
- `event_code` — one of the codes above.
- `timestamp` — 64-bit side-band sampled at emission.

Event-specific payload:

Event	Payload fields
hit	<code>set</code> , <code>way</code> , <code>state_before</code> , <code>state_after</code> , <code>we</code>
miss	<code>set</code> , <code>way</code> , <code>miss_class</code> , <code>we</code>
snoop	<code>snoop_type</code> , <code>hit</code> , <code>response_class</code>
eviction	<code>set</code> , <code>way</code> , <code>dirty</code> , <code>line_addr</code>
transition	<code>set</code> , <code>way</code> , <code>old_state</code> , <code>new_state</code> , <code>cause</code>
<code>fill_start</code> / <code>fill_end</code>	<code>line_addr</code>
<code>drain_start</code> / <code>drain_end</code>	<code>line_addr</code>
dropped	<code>dropped_count</code>

21.4 Miss-Class Encoding

Miss classes follow the `cache_sim` computation:

Code	Class	Meaning
<code>AMBER_MISS_COMPULSORY</code>	compulsory	First access to this line.
<code>AMBER_MISS_CAPACITY</code>	capacity	Line was evicted by capacity pressure.
<code>AMBER_MISS_CONFLICT</code>	conflict	Line was evicted by mapping conflict.

The golden model supplies the class; the RTL records what it is told. When the model is not present, the class is recorded as AMBER_MISS_UNKNOWN.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

22 Drop-and-Count Behavior and monbus_tally_axil

22.1 Drop-and-Count Policy

`amber_monlite` is drop-and-count, not stall-and-backpressure. This preserves measurement integrity: the observer cannot perturb the cache hit/miss timing it is trying to measure.

When a packet is emitted and `mon_ready` is low, the packet is discarded and a saturating counter increments. When `mon_ready` next goes high, if the counter is non-zero an `AMBER_EV_DROPPED` packet is emitted with the count, and the counter clears.

22.2 Counter Width

The drop counter is 8 bits. The probability of 256 consecutive drops in a correctly sized system is negligible; if it occurs, the counter saturates and the overflow is visible in the dropped packet.

22.3 monbus_tally_axil Integration

Amber's MonBus connects to the shared `monbus_tally_axil` agent exactly like `STREAM` and every other component in the repo. The agent:

1. Accepts `mon_valid/mon_ready`.
2. Parses the 128-bit packet using `monitor_common_pkg`.
3. Maintains per-event-class counters in AXI-Lite addressable registers.
4. Reports dropped-packet counts.

22.4 Software View

Software reads event counters through the `monbus_tally_axil` AXI-Lite slave. There is no APB CSR inside `amber`. The counter addresses and field layouts are defined by the tally agent, not by `amber`.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

23 What the K-maps Prove

23.1 Methodology

The kmap workbook (`gen_amber_contracts_kmaps.py`) applies the STREAM-derived methodology:

1. Identify every important combinational decision signal.
2. List axes as (name, expression, cite) triples.
3. State `depends_only_on` evidence that justifies the axis list.
4. Add checked relations that mark unreachable input combos as don't-cares.
5. Derive the minimal SOP mechanically with Quine-McCluskey.
6. Diff the derived cover against the RTL SOP when RTL exists.

Because amber has no RTL yet, the workbook is a **pre-RTL contract**. The derived SOP is the logic the future RTL must implement. `rtl_sop` is intentionally absent in the workbook; when RTL lands the diff step produces one of three verdicts.

23.2 The Three Verdicts

The verdict logic is implemented in `bin/kmaps/minimize.py`:

Verdict	Meaning	Action
IDENTICAL	The RTL SOP matches the derived minimal cover.	No change needed; the RTL is already minimal.
RTL-REDUNDANT	The RTL SOP includes extra literals or terms but produces the same truth table.	Document why the redundancy exists (timing, readability, defensive coding).
RTL-DIFFERS	The RTL SOP produces a different truth table than the derived cover.	Investigate: bug, unstated invariant, or missing relation.

23.3 First-Verdict Status (2026-10-06)

The first RTL slice landed: `amber_pkg` (the Table 3.0 decode functions `amber_snoop_crresp` / `amber_snoop_next_state`) and its module wrapper `amber_snoop_kmap`. Verdicts for the six snoop sheets (the four CRRESP maps and the two next-state maps):

- **Truth-table equivalence: PASS at gate/func/full.** The TB (`dv/tests/fub/test_amber_snoop_kmap.py`) carries an independent copy of HAS Table 3.0 and exhausts the reachable space at gate, adds the reserved encodings at func, and the whole 3-bit x 3-bit space at full. One real discrepancy was caught and resolved in the TB's favor of the documentation: Modified + CleanInvalid carries `IsShared=1` per Table 3.0.
- **SOP-literal diff against the QM-derived covers: NOT CHECKED, deferred.** The RTL implements the table as a case decode, not minimal SOP literals; the meaningful diff is the truth-table one above. Supplying `rtl_sop=` and re-deriving is deferred until `amber_snoop_resp` integrates this decode — if the integration restructures the logic, the literal diff would be repeated work.

All other sheets (address decode, control FSM, MonBus events) remain pre-RTL contracts with
VERDICT: NOT CHECKED.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

24 RTL Landing Diff Procedure

24.1 When RTL Lands

After the first RTL commit for a module, update the corresponding sheet in `gen_amber_contracts_kmaps.py`:

1. Add citations to the RTL file and line numbers in the CITES registry.
2. Replace `rtl_sop=None` with the actual RTL expression as a sum-of-products string.
3. Re-run the generator.
4. Review each sheet's verdict.

First landing (2026-10-06): the snoop CRRESP / next-state decode exists as `amber_pkg` functions + `amber_snoop_kmap`. See [01_kmap_verdicts.md](#) for why its sheets record truth-table PASS but a deferred SOP-literal diff; the procedure above applies to every sheet still at `rtl_sop=None`.

24.2 Diff Checklist

For every sheet that reports **RTL-DIFFERS**:

- Is the difference a deliberate redundancy? Re-label as RTL-REDUNDANT and add a note.
- Is an unstated invariant doing work? Add it as a checked relation and re-derive.
- Is the RTL expression wrong? File a bug and fix it.

For every sheet that reports **RTL-REDUNDANT**:

- Does the redundancy serve timing closure? Keep it and document.
- Does the redundancy serve readability? Keep it if the cost is small.
- Is it accidental? Simplify the RTL.

24.3 Regression Rule

The generator is part of the CI check for amber. Any RTL edit that moves or changes a cited expression must cause the generator to fail loudly, not silently publish a stale map. This is enforced by the CITES registry in `gen_amber_contracts_kmaps.py`.

Last Updated: 2026-10-06

25 gem5 FSM Cross-Check

25.1 Source

The Python reference model and the formal FSM oracles derive from the gem5 Ruby MESI_Two_Level SLICC tables, specifically `L1cache.sm`. The enumerated transient states in that file are the source for the `amber_pkg` state encoding and for the control FSM states in `amber_control`.

25.2 Cross-Check Mechanism

1. Extract every stable and transient state from `L1cache.sm`.
2. Map gem5 events to amber signals (CPU request, snoop type, fill response, drain done).
3. Replay a randomized trace through both the gem5-derived Python model and a future cycle-accurate RTL model.
4. Compare state transitions, CRRESP outputs, and next-state writes.

25.3 Scope

The cross-check covers:

- All M/E/S/I stable states.
- Transient states between stable states and pending responses.
- The six IHI0022 snoop types.
- The onyx D2 coherent issue subset (ReadShared, ReadUnique, CleanUnique, MakeUnique, WriteBack, Evict).

MOESI O-state is out of scope for v1.0; the 3-bit encoding reserves a code for it.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

26 Elaboration Configurations

26.1 Default Config

The default elaboration geometry is the PRD D1 center point:

Parameter	Value
SETS	128
WAYS	4
LINE_BYTES	64
BUS_WIDTH	64
REPL_POLICY	"LRU"
WRITE_POLICY	"wb_wa"
VICTIM_DEPTH	1

This is the configuration used for FPGA characterization, cache_sim parity, and the FULL cocotb grid.

26.2 Tiny Formal Config

The SymbiYosys control-layer proofs use a tiny geometry for tractability:

Parameter	Value
SETS	16
WAYS	2
LINE_BYTES	64
BUS_WIDTH	64
REPL_POLICY	"LRU"
WRITE_POLICY	"wb_wa"
VICTIM_DEPTH	1

The tiny config reduces tag-array depth and way count so that the formal tool can fully explore the control FSM and pending-fill bypass interactions.

26.3 Bring-Up Config

A minimal bring-up config may use:

Parameter	Value
SETS	4
WAYS	2
LINE_BYTES	32
BUS_WIDTH	32
REPL_POLICY	"FIFO"
WRITE_POLICY	"wt_na"
VICTIM_DEPTH	1

This is for initial module-level smoke tests only, not for characterization or formal proofs.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

27 DV Matrix Dimensions

27.1 Pattern-B Cocotb Grids

The primary DV layer is Pattern-B GATE/FUNC/FULL cocotb grids over the geometry × policy × rig matrix.

27.1.1 Dimensions

Dimension	Values
Geometry	default (128/4/64/64), tiny (16/2/64/64), bring-up (4/2/32/32)
Policy	LRU, FIFO, RANDOM, tree-PLRU
Rig	amber (pair-rig AXI4), amber_ace (onyx-rig ACE)
Write policy	wb_wa, wt_na (bring-up only)

27.1.2 Grid Coverage

Grid	Geometry	Policy	Rig	Purpose
GATE	tiny	LRU	amber	fast gate-level sanity
FUNC	default	LRU, FIFO, RANDOM	amber	functional regression
FUNC	default	LRU	amber_ace	coherent-transaction sanity
FULL	default	all four	both rigs	nightly regression + cache_sim parity
FULL	tiny	LRU	both rigs	formal companion runs

27.2 Formal Targets

SymbiYosys is scoped to the control layer at the tiny geometry:

- No stale data served after an external write (pending-fill bypass correctness).

- No deadlock or livelock under fair arbitration.
- Victim-buffer handoff correctness.
- `amber_snoop_resp` AC/CR/CD ordering.

27.3 Golden-Model Parity

For LRU, FIFO, and RANDOM, `amber` hit/miss/miss-class counts must match `cache_sim` at identical geometry and policy. `tree-PLRU` has no `cache_sim` golden model today; it is cross-checked by RTL self-checks and recorded as a future sim extension.

27.4 MonBus Coverage

Every event class in Chapter 4 must be visible on MonBus in simulation. The `_monlite` wrapper is present-vs-absent in the regression so the gate-cost delta is part of the evidence.

Last Updated: 2026-10-06

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

28 amber — Product Requirements (DRAFT)

Version: 0.5 (draft, 2026-10-06) **Status:** nine of the eleven decision rows record a name and a date; D3 (memory-side, now shaped by the two-top structure) and D11 (arrays, house primitives by rule) remain OPEN. A decision becomes DECIDED when it records a name and a date, the way the reed-solomon PRD does it. v0.2: D8 decided — *_monlite observation wrappers (never _mon on the measured paths). v0.3: D4 decided — snoop transport is ACE-shaped (AC/CD/CR) per the [onyx D7](#) port contract, family decision 2026-10-05. The snoop-filter (directory-lite) half of D4 stays deferred and is reopened as [onyx D3](#). v0.4: D2 decided (Sean, 2026-10-06) — CPU-side interface is a **GAXI slave**. House BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. v0.5 (Sean, 2026-10-06): D1, D5, D6, D7, D9, D10 decided in one working session — geometry 32 KiB/64 B/4-way center + a tiny 16-set/2-way formal config; write-back + write-allocate; plain MESI on a 3-bit state field; replacement {LRU, tree-PLRU, FIFO, RANDOM} with LRU default (tree-PLRU the timing fallback, no sim golden model — future sim extension recorded); verification = Pattern-B grids + control-layer SymbiYosys at tiny geometry + cache_sim parity on LRU/FIFO/RANDOM + FSM oracles derived from the gem5 Ruby MESI_Two_Level SLICC tables; first consumer = TB masters bring-up, pair rig the gated deliverable, STREAM attach the documented future hook.

28.1 1. Purpose

A parameterised, blocking, snoopy MESI L1 data cache for the Sherpa research/teaching flow: correct against a software golden model, provable with SymbiYosys, observable through the MonBus monitor fabric, and characterised for FPGA cost like every other component in this repo. It is the first cache-ip IP and the deliberate baseline for [jet](#), the lockup-free follow-on.

28.2 2. Background

A blocking cache freezes on a miss: the pending request (and everything behind it) waits for the fill to complete. That is poor for performance and excellent for correctness work — one outstanding transaction means no MSHR bookkeeping, no hit-under-miss hazards, no merge logic. MESI (Modified / Exclusive / Shared / Invalid) is the coherence layer on top: each line carries a state that says who may write it and who must be told when they hold a stale copy. Snoopy coherence broadcasts probes on a shared bus; every cache watches and answers.

Blocking and MESI are orthogonal, which is the design bet: amber nails the coherence protocol with trivial miss handling; jet keeps the protocol and upgrades the miss handling. The pair then answers the classic question with measured, on-board data: what does lockup-freedom actually buy?

Repo anchors the design must respect: resets on the ALWAYS_FF_RST macros, storage on the shared sdpram_core / gaxi FIFO primitives (no bespoke SRAM, no reset on memories), [DEPTH] array syntax, valid/ready streaming contracts, and the monitor/MonBus observation fabric from rtl/amba/.

28.3 3. Decisions that pick the design

#	Decision	Candidates	What it drives
D1	Cache geometry	DECIDED 2026-10-06 (Sean): 32 KiB / 64 B lines / 4 ways (128 sets) as the default center, fully elaboration-parameterised (4–32 KiB, 32–64 B, 2–8 ways), plus a reserved tiny config — 16 sets / 2 ways / 64 B — for SymbiYosys tractability. Board BRAM does not bind (~9 of 135 36Kb RAMs on Nexys A7 at the default), so geometry serves the DV matrix and the proofs, not the board	tag/data array dimensions, the whole DV matrix
D2	CPU-side interface	DECIDED 2026-10-06 (Sean): GAXI slave. Chosen over the plain valid/ready native port and the AXI4 slave: house BFM/monitor coverage and	front-end FSM, adapter surface, first consumer

#	Decision	Candidates	What it drives
		skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. Must express single- beat reads, write- allocate fills, and (if write-through is chosen) store- without-allocate	
D3	Memory-side interface	OPEN: AXI4 read/write masters on the house axi4_master_rd/w r wrappers (stream/rapids pattern) vs GAXI vs a simple direct SRAM port for bring-up	fill/drain engines, burst behavior on misses
D4	Snoop transport	DECIDED 2026- 10-05 (Sean): ACE-shaped transport — amber implements the AC/CD/CR snoop channels exactly as onyx D7 defines them (family decision: cache-ip is ACE-	the coherence bus, formal surface, gate count

#	Decision	Candidates	What it drives
		shaped), staying bus-agnostic internally behind a thin adapter. Chosen over a custom dedicated snoop bus and over snooping on the AXI4 fabric directly; the ACE-lite-shaped candidate was set aside because a snoop responder must drive CR responses and CD data — ACE-Lite carries no snoop channels at all. The snoop-filter (directory-lite) sub-question stays deferred, reopened as onyx D3.	
D5	Write policy	DECIDED 2026-10-06 (Sean): write-back + write-allocate. Write-through/no-allocate deletes the coherence traffic the research exists to measure — dirty	M-state logic, memory traffic, formal targets

#	Decision	Candidates	What it drives
		snoop transfers (PassDirty), WriteBack drains, onyx's memory-read elision — and leaves MESI's M state with no job; it stays available as a bring-up mode only	
D6	MESI variant	DECIDED 2026-10-06 (Sean): plain MESI on a 3-bit state field. MOESI's O-state is a later elaboration upgrade (encoding headroom reserved now, ~512 extra tag bits); its win — dirty→shared without a memory read — is a measurement for the pair rig, not a v1 requirement	state machine count, snoop response matrix
D7	Replacement policy	DECIDED 2026-10-06 (Sean): policy set {LRU, tree-PLRU, FIFO, RANDOM}, default LRU.	policy engine, cross-check fidelity

#	Decision	Candidates	What it drives
		LRU/FIFO/RANDO M match the cache simulator exactly (POLICIES = ['LRU' , 'FIFO' , 'RANDOM']), so golden-model parity covers every policy both sides implement; tree-PLRU is carried as the timing-tight fallback — the one policy with no sim golden model, cross-checked by RTL self-checks today and a future cache_sim extension (recorded, not silently assumed). Policy stays an elaboration parameter so the sim ↔ RTL cross-check holds per policy	
D8	Observation	DECIDED 2026-10-04 (Sean): *_monlite wrappers, not the heavyweight	perf counters, board capture, DV scoreboards

#	Decision	Candidates	What it drives
		<u>_mon</u>. MonBus taps for hits, misses (class split if the model supplies it), snoops, evictions, and state transitions, observed through the axi4_monlite / axil4_monlite family — same 128-bit MonBus packets, UNIT/AGENT ids, and host tooling as _mon, ~1/5 the monitor gates, and a drop-and-count policy that never stalls the port. Rationale for a research cache: the observer must not perturb what it measures — a <u>_mon</u> wrapper that gates the port when its tracking tables fill would corrupt the very miss-latency numbers amber and jet exist to produce. The	

#	Decision	Candidates	What it drives
		events D8 lists are emitted as MonBus packets and tallied by the standard agents (monbus_tally_axil); heavyweight _mon stays available as a DV cross-check, never on the measured paths.	
D9	Verification strategy	DECIDED 2026-10-06 (Sean): Pattern-B cocotb GATE/FUNC/FULL grids over the geometry × policy × rig matrix (ACE BFM: cocotb-framework 1.2.0); SymbiYosys scoped to the control layer — amber_control, amber_snoop_resp, the pending-fill bypass, the victim-buffer handoff — proven at the tiny 16-set/2-way geometry; cache_sim trace-	DV structure, formal areas, CI time

#	Decision	Candidates	What it drives
		replay parity across LRU/FIFO/RAND OM. The Python reference model and the FSM oracles derive from the gem5 Ruby MESI_Two_Level SLICC tables in ../References/ge m5-ruby- protocols/ — L1cache.sm's enumerated transient states are the derivation source for the amber_pkg state encoding, and every stable and transient transition is cross-checked against an executable protocol spec, not prose	
D10	First consumer	DECIDED 2026-10-06 (Sean): standalone TB masters for bring-up; the gated deliverable	integration scope, what “done” means

#	Decision	Candidates	What it drives
		is the pair rig — two ambers + shared memory, the coherence research rig. STREAM attach is documented as the future integration hook (D2's GAXI choice is what makes it cheap later); the onyx rig is sequenced per onyx D10 — onyx waits behind the pair rig	
D11	Data/tag arrays	OPEN (direction: shared primitives): sdpram_core- based tag/data stores + house FIFOs for pending/fill queues; FPGA attributes per ../.../.../GLOB AL_REQUIREMENTS. md	area/timing headroom, the no- bespoke-SRAM rule

28.4 4. Success criteria

1. **Golden-model parity.** Given the same trace + policy + geometry, amber's hit/miss/miss-class counts match `cache_sim` exactly across a regression trace suite.

2. **Proofs.** SymbiYosys: no stale data is ever served after an external write; the coherence protocol cannot deadlock under fair arbitration.
3. **Observation.** Every event class above visible on MonBus in simulation, and captured on at least one FPGA board.
4. **Cost.** Synthesis/implementation numbers (LUT/FF/RAM, fmax) on Nexys A7 and/or Genesys 2, recorded in docs/ the way pumice characterised.

28.5 5. Non-goals

Non-blocking / lockup-free miss handling (jet), directory-based coherence, L2/L3 or multi-level inclusion policies, ECC on cache arrays, virtual memory and TLB interaction, multi-port CPU issue. Each is a future IP or a future decision, not an amber requirement.

28.6 6. Where the block sits

Research cache on the AMBA fabric: CPU-side slave (D2), memory-side AXI4 masters (D3), snoop port toward onyx / peer caches (D4 — ACE-shaped AC/CD/CR, decided 2026-10-05), MonBus observation (D8). Standalone-testable with TB masters before any consumer exists (D10).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

29 amber — Pre-Hardware Architecture Specification (Pre-HAS)

Version: 0.3 (draft, 2026-10-06) — **superseded by the full HAS (v1.0, same day)** once the PRD closed its rows (v0.5): the chapter book ratifies this sketch, and this file is kept as the decision-path record. **Status:** architecture sketch written while the PRD decisions are still OPEN. This document records **no decisions**. “PROPOSED” marks a working default put forward so the decision rows can be argued with concretely; a decision becomes DECIDED only when [PRD.md](#) records a name and a date. When the decisions close, each PROPOSED item here is either ratified into the full `amber_has` chapter book (the stream/andesite convention) or overridden and this file edited. v0.2 (2026-10-06): D2 landed in the PRD — CPU-side is a **GAXI slave**. F9 now answers probe-during-fill with a **pending-fill bypass** instead of an unbounded stall. Q1 answered — `amber_fabric` carries **both** rig attachments and the RIG elaboration parameter `generate-selects`. v0.3 (2026-10-06): Q1 re-answered by the owner — both rigs as **two tops**, `amber` (pair rig) and `amber_ace` (onyx rig), on a shared `amber_core`. No RIG parameter: the rig is whichever top you instantiate.

Governing documents: [PRD.md](#) (binding) · [onyx PRD D7](#) (snoop-port contract) · [jet PRD](#) (what must stay constant) · [GLOBAL_REQUIREMENTS.md](#) · [cache simulator](#) (golden model)

29.1 1. Intent of this document

The PRD carries eleven decision rows and has closed three: **D2** (CPU-side is a GAXI slave), **D4** (snoop transport is ACE-shaped AC/CD/CR), and **D8** (observation through `*_monlite` wrappers only). The Pre-HAS makes the open rows concrete: for each, it sketches the feature the way a working default would build it, so the architecture can be reviewed before any RTL exists. It is deliberately cheap to change — one file, no chapter skeleton yet.

29.2 2. Feature sketches

29.2.1 2.1 Core datapath — the blocking bet

- **F1 — Blocking single-outstanding pipeline.** A miss freezes the front end until the fill completes and the original request replays. No MSHR, no hit-under-miss, no merge logic — that deletion is the entire point; it is jet’s delta (jet J1–J5).
- **F2 — Geometry as elaboration parameters** (D1 OPEN). PROPOSED default profile 32 KiB / 64 B lines / 4 ways (128 sets) as the DV matrix centre of gravity; ranges 4–32 KiB, 32–64 B, 2–8 ways.

- **F3 — Pluggable replacement** (D7 OPEN). PROPOSED: true-LRU as the correctness reference (the cache simulator models it exactly), tree-PLRU as the FPGA-area variant; FIFO/random kept because the sim models them too. Policy is an elaboration parameter, never a code hack, so the sim ↔ RTL cross-check holds for every policy both sides implement.
- **F4 — Write policy** (D5 OPEN). PROPOSED: write-back + write-allocate — MESI's M state earns its keep, and dirty-victim handling is where blocking simplicity pays. Write-through/no-allocate retained as a bring-up mode because it deletes the victim path entirely.
- **F5 — Whole-line fills.** One burst per fill; sub-line fill granularity is a non-goal. Burst length derives from $\text{LINE_BYTES} / (\text{BUS_WIDTH}/8)$.

29.2.22.2 Coherence

- **F6 — MESI state per line** (D6 OPEN). PROPOSED plain MESI; MOESI's O-state recorded as a later elaboration upgrade, not v1 — ownership's win (dirty-transfer reduction between caches) is a measurement for the pair rig, not a v1 requirement. Insurance: allocate a 3-bit state field now (MESI needs 2; MOESI needs 3) so the upgrade is an encoding change, not an array rebuild.
- **F7 — ACE snoop responder** (D4 DECIDED — sketched, not re-decided). `amber_snoop_resp`: a thin adapter speaking raw ACE AC/CR/CD on the outside per the onyx D7 contract (IHI 0022 the definition) and driving a bus-agnostic probe interface inside. The cache core never contains ACE — only the adapter does; that is exactly why D4 was chosen (onyx, or a real ACE interconnect, can supersede the custom bus without touching cache RTL).
- **F8 — Snoop/local arbitration — the culsans problem.** Probes must not starve and CPU tag latency must stay bounded. PROPOSED: tag array on the dual-port `sdpram_core` — port A for CPU lookups, port B for snoop lookups; data array port A for CPU/fill, port B for snoop readout feeding
 CD. No arbitration stall in the common case; the *control* collision (miss FSM vs snoop FSM wanting the same array control port) resolved with snoop-priority round-robin inside `amber_control`.
- **F9 — Probe-during-fill, answered by a bypass** (direction set 2026-10-06: add the bypass). Blocking means a fill is usually in flight, and a probe against a pending line must not observe pre-fill state. The answer is not an unbounded stall: amber keeps one pending-fill register — single

outstanding by construction — recording {line address, state being installed, data availability}. A probe matching it is answered from the register: CRRESP reflects the post-fill state, and required CD beats are forwarded from the fill stream as its beats arrive. A probe that misses it takes the normal tag path. The snoop response is bounded either way. The correctness target sharpens rather than relaxes: *the bypass must be state-accurate* — a probe against a pending fill observes exactly the post-fill state, never pre-fill — alongside the response-bound liveness target. jet reopens the generalisation (jet J4/J6): multiple outstanding fills turn the register into a pending-fill table, which is jet's problem, not amber's.

- **F10 — CRRESP response matrix** per MESI state, mirroring the RDS-DV ACE BFM default matrix as corrected in cocotb-framework 1.2.0 (MakeInvalid on M/O invalidates without data transfer — the bug the release fixed). The matrix ships as an explicit table in the full HAS.

29.2.32.3 Fabric side — where the two deployment rigs meet

- **F11 — The two rigs.** *Pair rig* (amber D10): two ambers + shared memory, snoops peer-to-peer on the D4 port, misses are plain AXI4 to memory. *Onyx rig*: misses become coherent transactions toward onyx; onyx owns the memory side and drives the snoops. One cache core, two fabric attachments.
- **F12 — Fabric side: two tops on one core** (Q1 answered 2026-10-06: do both, as two tops). The rig-agnostic cache core is shared; the fabric difference is confined to the top wrapper and its master stack:
 - **amber** — the pair-rig top. Misses are plain AXI4 to memory on the house `axi4_master_rd/wr` wrappers; snoop encodings never driven; the ACE snoop-responder port faces the peer cache.
 - **amber_ace** — the onyx-rig top. Fabric side is the existing `rtl/amba/ace/axi4ace_master_rd / axi4ace_master_wr` modules (ACE fields and RACK/WACK auto-pulse already in place), driven by a thin `amber_ace_issue` coherent-transaction issuer mapping cache events to the onyx D2 subset — read miss → ReadShared / ReadUnique (by desired exclusivity), write promotion → CleanUnique / MakeUnique, dirty eviction → WriteBack, clean eviction → Evict. Memory masters not exposed; the snoop-responder port faces onyx. No parameter selects between them: the rig is whichever top is instantiated. Both tops are compiled, and both are

exercised in the DV matrix — amber against a peer + memory environment, amber_ace against an onyx environment (§5).

- **F13 — Fill/drain engines** (D3 OPEN, direction: house wrappers). Fill = one AR burst, whole line. Drain = a small victim buffer (PROPOSED depth 1)
 - one AW burst. Single outstanding by construction — no reorder buffers.

29.2.4 2.4 Observation (D8 DECIDED — sketched, not re-decided)

- **F14 — amber_monlite wrapper.** MonBus event set: hit (read/write), miss with class split (cold/capacity/conflict, when the golden model supplies the class), snoop (type, hit/miss, response class), eviction (clean/dirty), MESI state transitions, fill/drain start+end. Tallied by monbus_tally_axil exactly like every other component; the wrapper never stalls the observed port — drop-and-count. The observer’s own gate cost is measured present vs absent (F18), so “the observer does not perturb what it measures” is a quantified claim, not an assertion.

29.2.5 2.5 Verification & formal surface (D9 OPEN — direction proposed)

- **F15 — Pattern-B cocotb grids.** GATE/FUNC/FULL over the geometry × policy × rig matrix. Instruments already exist: the RDS-DV AXI4ACESnoopMaster BFM drives amber’s snoop port and AXI4ACEMasterRead/ Write drive the fabric side (cocotb-framework 1.2.0).
- **F16 — Golden-model parity.** Trace replay from bin/apps/cache_sim/ at identical geometry + policy; hit/miss/miss-class counts must match exactly (PRD success criterion 1). The replacement policy parameter (F3) is what keeps this check meaningful across policies.
- **F17 — SymbiYosys targets** (PRD criterion 2): no stale data served after an external write (F9 ordering); no deadlock/livelock under fair arbitration. Proof surfaces named now so the RTL is built proof-shaped: amber_control FSM, amber_snoop_resp, tag-array port sharing, victim buffer handoff.

29.2.6 2.6 Cost & characterisation (PRD criterion 4)

- **F18 — Synthesis sweep.** Geometry params × replacement policy on Nexys A7 and/or Genesys 2; LUT/FF/RAM/fmax recorded in docs/ the way pumice characterised. Includes the F14 observer-present vs absent delta.

29.3 3. Proposed block architecture

Proposed module set (amber_* prefix, per the cache-ip convention):

Module	Role	PRD anchor
amber	Pair-rig top: amber_core + plain AXI4 master stack + snoop responder toward the peer	D10
amber_ace	Onyx-rig top: amber_core + ACE master stack + coherent-transaction issuer + snoop responder toward onyx	D10, F12
amber_core	Shared rig-agnostic core, instantiated by both tops	—
amber_cpu_frontend	GAXI slave (D2 decided); request accept, response return, replay latch	D2
amber_control	Blocking pipeline FSM: lookup → hit / miss → evict → fill → replay; owns tag-array control arbitration (F8) and the pending-fill bypass register (F9)	D1, D5
amber_tag_array	sdpram_core tag + state store (3-bit state field, F6)	D11
amber_data_array	sdpram_core data store	D11
amber_repl	Pluggable replacement (policy elaboration parameter)	D7
amber_victim	Dirty victim buffer + drain trigger	D5
amber_fill / amber_drain	AXI4 read/write engines; plain axi4_master_rd/wr	D3

Module	Role	PRD anchor
amber_snoop_resp	under amber, axi4ace_master_rd/wr under amber_ace ACE AC/CR/CD adapter; bus-agnostic inside — faces the peer in amber, onyx in amber_ace	D4
amber_ace_issue	Coherent-transaction issuer for the onyx D2 subset — amber_ace only	F12
amber_monlite	Observation wrapper (D8)	D8

Miss data flow: frontend accept → tag lookup (port A) → miss → amber_repl picks victim → dirty? amber_victim + drain engine WB → fill engine AR burst → data array write + state install → original request replays from the frontend latch → response.

Snoop data flow: AC arrives at amber_snoop_resp → tag lookup (port B) → line state determines CRRESP and whether CD beats follow → response driven. Conflict with an in-flight fill resolves per F9.

Observation taps sit on control-FSM decision points, not the datapath — the F8 port split means the datapath need not be touched to observe it.

29.4 4. Interface sketches (shapes, not pinouts)

Side	Sketch	Anchor
CPU-side	DECIDED (PRD D2, 2026-10-06): GAXI slave — house BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. Must express single-beat reads, write-allocate fills, byte enables.	D2

Side	Sketch	Anchor
Fabric-side	Two tops, shared core (F12): amber = plain AXI4 masters to memory + AC/CR/CD snoop responder toward the peer; amber_ace = axi4ace_master_rd/wr + amber_ace_issue toward the CCU (onyx D2 subset: ReadShared / ReadUnique / CleanUnique / MakeUnique / WriteBack / Evict) + the same snoop responder toward onyx. The rig is the top you instantiate — no mode parameter.	D3, D4
Observation	MonBus through amber_monlite; no other debug surface.	D8
Register block	None. Parameter-only IP; all visibility through MonBus tallies, unlike every other component's APB CSR. Recorded explicitly so it is a decision and not an accident.	—

29.5 5. Parameter sketch (illustrative — feeds the DV matrix)

Parameter	PROPOSED default	Range / set
SETS	128	16–512
WAYS	4	2–8

Parameter	PROPOSED default	Range / set
LINE_BYTES	64	32–64
REPL_POLICY	lru	lru, tree_plru, fifo, random
WRITE_POLICY	wb_wa	wb_wa, wt_na
VICTIM_DEPTH	1	1–2
BUS_WIDTH	64	32–128

Rig is **not** a parameter: it is the top you instantiate — `amber` or `amber_ace` (F12). Derived: `tag width = ADDR_WIDTH - $clog2(SETS) - $clog2(LINE_BYTES) + 3` (state field, F6); `fill beats = LINE_BYTES / (BUS_WIDTH/8)`.

29.6 6. Open questions this sketch surfaces

#	Question	Where it resolves
Q1	One mode parameter <code>amber_fabric</code> vs separate adapters per rig — RESOLVED 2026-10-06 : two tops on a shared <code>amber_core</code> — <code>amber</code> (pair rig) and <code>amber_ace</code> (onyx rig); no rig parameter, both compiled, both in the DV matrix (F12)	Full HAS implements it
Q2	GAXI vs plain valid/ready CPU side — RESOLVED by PRD D2 (2026-10-06, Sean): GAXI slave	<code>amber_cpu_frontend</code> RTL
Q3	Geometry default vs Nexys A7 RAM budget (F2)	With the F18 cost sweep
Q4	3-bit state field headroom for MOESI (F6)	Cheap now, expensive later — recommend taking the insurance

#	Question	Where it resolves
Q5	Probe during fill stall-bound — RESOLVED 2026-10-06: add the pending-fill bypass (F9); formal targets become bypass state-accuracy + response bound	Full HAS implements it; PRD criterion 2 depends on it
Q6	Replacement default: LRU reference vs tree- PLRU area (F3)	D7 closure, informed by F18

29.7 7. What the full HAS will add

The chapter book per the stream/andesite convention: front matter; purpose, conventions, definitions; use cases / key features / system context; graphviz block diagram; data flow; pin-level interfaces; performance targets; integration (clocking, reset, verification). The MAS follows when micro-architecture decisions land (array banking, pipeline staging). This Pre-HAS is the draft those books grow from — sections 2–6 above map onto chapters 2–5.

29.8 8. Cross-references

- [amber PRD](#) — binding requirements; decision record
- [onyx PRD D7/D2](#) — snoop-port contract and coherent-transaction subset the ONYX rig speaks
- [jet PRD](#) — J-rows hold amber constant; nothing here may silently pre-decide jet (F9's stall policy is amber's; jet J4/J6 explicitly reopen it)
- [References/AMBA_ACE_Interface_Definition.md](#) and [References/gem5-ruby-protocols/](#) — protocol definition and golden-protocol source
- RDS-DV cocotb-framework 1.2.0 ACE BFM — the DV instruments F15 assumes

Last Updated: 2026-10-06 **Maintained By:** amber architecture (pre-decision working document)

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

30 jet — Product Requirements (DRAFT)

Version: 0.1 (draft, 2026-10-04) **Status:** scaffold only — inherits every DECIDED row of [amber's PRD](#). Decided inherited rows so far: D8 observation (*_monlite, 2026-10-04) and D4 snoop transport (ACE-shaped AC/CD/CR per the onyx D7 contract, 2026-10-05). The rows below are the deltas jet must decide for itself; all are OPEN until they record a name and a date.

30.1 1. Purpose

The lockup-free (non-blocking) MESI snoop L1 cache: amber with MSHRs. The research question is measurement — with coherence, arrays, and observation held constant, what do outstanding misses, hit-under-miss, and miss merging actually buy in latency and bandwidth, in simulation and on board?

30.2 2. Decisions jet adds to amber's

#	Decision	Candidates	What it drives
J1	MSHR organization	OPEN: full associativity keyed by address vs direct-mapped by line index vs CAM of pending lines; capacity 2–16 entries as an elaboration parameter	the miss queue, merge logic, formal surface
J2	Hit-under-miss scope	OPEN: hits to any resident line (default) vs hits only to lines not conflicting with a pending fill vs reads-only (writes still block)	hazard logic between fill port and hit port
J3	Miss merging policy	OPEN: merge only exact-line aliases (default) vs merge line + offset range	memory traffic, MSHR occupancy, fairness

#	Decision	Candidates	What it drives
J4	Ordering guarantees	vs no merging (issue duplicate fetches, dedupe on return) OPEN: CPU-side responses in-issue-order vs out-of-order with ID tags; snoop-during-miss handling (a probe against a pending line must see the post-fill state or stall cleanly)	response path complexity, protocol proofs, consumer expectations
J5	Eviction under misses	OPEN: a dirty victim behind an outstanding fill must write back without deadlocking against the fill; victim buffer depth (0 = block like amber, 1–4 = decouple)	write-back path, the classic lockup-free deadlock corner
J6	Congestion/fairness	OPEN: policy when MSHRs are full — stall new misses (strict) vs drop-and-retry vs throttle CPU-side accepts; plus snoop-vs-fill arbitration	starvation/liveness proofs

#	Decision	Candidates	What it drives
J7	The measured claim	priority OPEN: define the experiment now — same trace suite + same board as amber, report hit latency distribution, miss concurrency, bandwidth, and MSHR occupancy from MonBus capture	what “jet is faster by X” must mean, reproducibly

30.3 3. Success criteria (additions to amber’s)

1. **Correctness inheritance.** Every amber proof and golden-model parity result still passes with MSHRs present (proofs are re-run, not assumed).
2. **The delta, measured.** On the amber trace suite and at least one board: reported latency/bandwidth improvement with MonBus evidence, and the cache_sim cross-check extended so the model predicts jet’s concurrency numbers too (or the model gap is documented as a finding).
3. **The deadlock corner, proven.** J5/J6 formal targets: no deadlock when MSHRs are full and dirty victims wait behind fills.

30.4 4. Non-goals (for jet, beyond amber’s)

Speculative prefetching, adaptive replacement, quality-of-service policies, multi-banked parallelism beyond what MSHRs imply. Each is a future IP or a later decision.

31 onyx — Product Requirements (DRAFT)

Version: 0.3 (draft, 2026-10-05) **Status:** decisions pending — this page records the questions and the candidates, not answers. A decision becomes DECIDED when it records a name and a date, the way the reed-solomon PRD does it. Until then every row below is OPEN. v0.2: D7 decided 2026-10-05 (Sean) — onyx owns the ACE-shaped AC/CD/CR snoop port; amber/jet adapt to it behind a thin adapter. Decided together with [amber D4](#), so the gate between the two is resolved. The sequencing constraint that remains: onyx still waits behind amber’s pair rig (amber D10), its first consumer. v0.3: D10 direction tightened 2026-10-05 — the ACE BFM extension of the RDS-DV axi4 family is now specified concretely (layering, four classes, build order); the row stays OPEN. RDS-DV implementation filed as [RTLDesignSherpa-DV issue #86](#).

31.1 1. Purpose

A snoop-based cache coherency unit (CCU) in the shape of Arm ACE: the manager that sits between the repo’s coherent caches and memory on the house AXI4 fabric. An attached cache (amber instance, jet instance) is an ACE *master*; it issues coherent transactions (ReadShared, ReadUnique, CleanUnique, MakeUnique, WriteBack, Evict) instead of managing peers directly. onyx receives those transactions, broadcasts a snoop (AC request) to the other caches, gathers their CR responses (DataTransfer / IsShared / PassDirty) and CD data, decides whether the transaction may be completed from cache-supplied data alone or memory must be read, and serialises same-address traffic. The research question is the manager side itself: what do fanout, gather, adjudication, and ordering cost — in gates, in latency, and in verification effort.

31.2 2. Background

ACE is AXI4 plus three snoop channels (AC, CR, CD) — and this repo already speaks AXI4 (skid buffers, GAXI FIFOs, `axi4_master_rd/wr` wrappers, Pattern-B BFM). The natural-protocol argument: every piece of onyx’s data path reuses existing, proven plumbing; the new logic is the coherence decision layer and the serializer. The definition is Arm IHI 0022 (the ACE chapters); the worked example is `pulp-platform/culsans` (ETH/PlanV), whose two halves are exactly the reading list:

- **culsans’ CVA6 coherent WB cache** — how a cache becomes a snoop responder: the AC/CD/CR channels, how snoop tag lookups arbitrate with the core’s own tag port, and the MOESI-ish line states.
- **ace_ccu_top** — the manager: broadcast snoops on a coherent read/write, collect responses, decide whether memory is read, and serialise same-address transactions.

Treat `culsans` as a worked example, not a dependency (last push 2024, lightly maintained). Note also what onyx is *not*: it is not a directory, not an L2, and not a general NoC — amber’s deferred “snoop filter / directory-lite” question (amber D4) shows up here only as one candidate row (D3).

Repo anchors the design must respect: resets on the ALWAYS_FF_RST macros, no bespoke SRAM (shared sdpram_core / house FIFO primitives), [DEPTH] array syntax, valid/ready streaming contracts, MonBus observation through *_monlite wrappers only (amber D8's decision — the observer must never stall the measured paths), filelists registry, and GLOBAL_REQUIREMENTS.

31.3 3. Decisions that pick the design

#	Decision	Candidates	What it drives
D1	Functional scope	OPEN: CCU only — N ACE slave-side ports toward caches, one AXI4 master toward memory, one ACE-Lite IO port (the culsans shape; recommended starting scope) vs CCU + address striping / multiple memory ports (interconnect territory; non-goal today)	block boundary, what “onyx done” means
D2	ACE transaction subset	OPEN: research subset — ReadShared / ReadUnique / CleanUnique / MakeUnique / WriteBack / Evict (recommended) vs subset + barrier support vs full ACE incl. DVM	front-end decoder, BFM scope, conformance matrix
D3	Snoop fanout	OPEN: broadcast to all attached caches (culsans	fanout logic, gate count, the coherence-traffic

#	Decision	Candidates	What it drives
		shape, simplest, recommended for v1) vs directory-lite snoop filter as an elaboration parameter (amber D4's deferred question, reopened here) vs snoop filter mandatory	measurements
D4	Response gather	OPEN: fixed response window (all caches respond within N beats, presumed-miss after timeout — simple, needs a proven bound) vs tagged outstanding snoop IDs with per-transaction completion (exact, more bookkeeping)	gather buffer, formal surface, ACE conformance
D5	Same-address serialization	OPEN: per-address scoreboard allowing interleaving of different addresses (recommended; the interesting manager) vs full	the serializer, the deadlock/liveness proof, jet J4 interactions

#	Decision	Candidates	What it drives
		in-order single outstanding (simplest, strangles concurrency; fine as a bring-up mode)	
D6	Memory-read elision	OPEN: full adjudication — read memory only when no cache supplied data and no PassDirty (correct ACE semantics; recommended) vs always-read simplification for bring-up	memory traffic, the measurable win onyx exists to quantify
D7	Cache-side interface ownership	DECIDED 2026- 10-05 (Sean): onyx defines and owns an AC/CD/CR-shaped snoop port; amber/jet stay bus-agnostic internally and add a thin adapter to it. Chosen over caches natively growing ACE ports — the cache cores stay unpinned from	the amber D4  onyx contract, who changes when ACE supersedes the custom bus

#	Decision	Candidates	What it drives
		ACE except at the adapter, so onyx (or a real ACE interconnect, e.g. culsans' ace_ccu_top) can supersede the custom bus without touching cache RTL.	
D8	Domain / barrier rules	OPEN: single shareability domain, barriers ignored (recommended non-goal, recorded here so it is a decision and not an accident) vs minimal barrier ordering vs full DVM	scope honesty, later upgrade path
D9	Observation	OPEN (direction: amber D8 pattern): *_monlite taps for coherent-txn start, snoop fanout per transaction, response gather completion, memory-read elision hit/miss, serialization stalls, and	perf counters, board capture, DV scoreboard

#	Decision	Candidates	What it drives
D10	Verification strategy	WB/Evict drains; tallied by monbus_tally_axi4 OPEN (direction: Pattern-B cocotb gate/func/full. ACE BFM — an extension of the axi4 BFM family in RDS-DV, layered the way that family is built (analysis from inspecting CocoTBFramework/components/axi4/): (1) field configs — an AXI4ACEFieldConfigHelper adds ARSN00P[3:0] / AWSN00P[2:0] to the AR/AW field configs; (2) signal-map patterns — new axi4ace_* protocol patterns (decide there: framework-shipped vs. user-loadable, the signal_mapping_helper question); (3) transaction/packet layer — the six	DV structure, formal areas, CI time

#	Decision	Candidates	What it drives
		D2 transaction types as an enum, CRRESP as a bitfield (DataTransfer/Error/PassDirty/IsShared/WasUnique); (4) four BFM classes — AXI4ACEMasterRead / AXI4ACEMasterWrite as subclasses (snoop type on commands; the BFM drives RACK/WACK when playing master against an onyx-port DUT, since the RTL auto-pulse in axi4ace_master_read/wr only covers DUT-as-master), AXI4ACESnoopSlave (new, no base: receives AC, drives CR/CD in issue order — snoops carry no ID — with programmable cache-state behavior producing the correct CRRESP;	

#	Decision	Candidates	What it drives
		this is the golden-model hook that replays gem5 traces through amber), AXI4ACESnoopMaster (new: drives AC, gathers CR/CD in order — drives onyx’s fanout in DV); (5) compliance checker — in-order CR/CD, no snoop to the requesting master, CD-before-CDLAST, CRRESP validity per snoop type; (6) main-repo TB classes under bin/TBClasses/axi4ace/ per consumer. Build order: snoop-slave BFM first (amber’s pair rig needs it), snoop-master BFM lands with onyx; ships as an RDS-DV framework release the way the irq BFM did. Python golden	

#	Decision	Candidates	What it drives
		model derived from the gem5 Ruby MESI_Two_Level spec in ../References/gem5-ruby-protocols/; the ACE subset contract is the in-repo definition doc; SymbiYosys proofs of same-address serialisation and no-deadlock under fair arbitration; culsans behaviours as informal cross-checks, not oracles)	
D11	Master count & profile	OPEN: N coherent cache masters 2–4 as an elaboration parameter (culsans uses 2–4 CVA6) vs fixed 2 for the research rig; line size / profile locked to amber’s geometry vs independently parameterised	array dimensions, DV matrix, the measured system’s realism

31.4 4. Success criteria

1. **Three-block coherence.** Two amber instances (or amber + jet) plus onyx form a coherent system that passes the golden-model parity suite amber D9 defines — the pair rig grown by one block, same traces, same policy space.
2. **IHI 0022 conformance on the D2 subset.** A scoreboard checks the AC/CR/CD behaviour per transaction type: the CR bits (DataTransfer, IsShared, PassDirty) of every gather are correct for the transaction and the caches' states.
3. **The serializer, proven.** Formal: no two same-address coherent transactions proceed incompatibly; no deadlock under fair arbitration at $N \in \{2, 3, 4\}$ masters; the D4 response-gather mechanism cannot livelock (a presumed-miss bound exists and is proven).
4. **Observation.** Every D9 event class visible on MonBus in simulation and captured on at least one FPGA board.
5. **Cost.** Synthesis/implementation numbers (LUT/FF/RAM, fmax) on Nexys A7 and/or Genesys 2, recorded in docs/ the way pumice characterised.

31.5 5. Non-goals

Cache storage in onyx (it is a manager, not an L2 — tempting because a CCU's gather point is the natural place to cache; explicitly out), directory-based coherence (the D3 snoop-filter candidate is the closest this IP comes), ACE DVM and full barrier semantics (D8), cache stash/dealloc features, multi-CCU / multi-domain systems, and any CPU-side or upstream-master port beyond ACE-Lite IO. Each is a future IP or a future decision, not an onyx requirement.

31.6 6. Where the block sits

The fabric's coherence point, on three kinds of port: **cache-side**, N ACE-facing ports accepting coherent transactions from amber/jet instances acting as ACE masters (D7, decided 2026-10-05: onyx owns this port's ACE shape; caches adapt); **memory-side**, one AXI4 master toward the house memory path (D6 fixes when it is read); **inter-cache**, the snoop fanout — AC requests driven into each peer cache, CD snoop data and CR snoop responses returned. No CPU port, no register block beyond what observation needs (D9 tallies live in `monbus_tally_axil` like every other component). Standalone-testable with BFM caches long before amber exists: culsans' `ace_ccu_top` demonstrates exactly that decoupling.

32 AMBA ACE — Working Interface Definition for cache-ip

Distilled 2026-10-05 for the cache-ip family, then **verified clause-by-clause against IHI 0022H on 2026-10-05** (Arm’s direct PDF, linked in §7; fetched to /tmp for comparison, not stored in-tree). Verification confirmed every transaction type, channel payload, CRRESP bit, and the ACE-Lite channel exclusions; it caught and fixed two errors in the ordering rules (the original draft claimed ACE serializes snoops one-at-a-time and mandates CD-before-CR — ACE requires only per-channel ordering and permits multiple outstanding snoops; the stricter behavior is recorded above as our own adapter convention). This document is the in-repo definition of the **ACE-shaped port contract** referenced by [amber D4](#) and [onyx D7](#) (family decision 2026-10-05: cache-ip is ACE-shaped).

Normative reference: Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022 (ACE chapters). Available from Arm without charge (direct PDF in §7); Arm copyright, so deliberately not mirrored here. Where this document and IHI 0022 disagree, IHI 0022 wins — file a docs bug.
Worked example: `pulp-platform/culsans` (`ace_ccu_top`, Solderpad license) — read its RTL after this page, not before.

32.1 1. What ACE is

ACE (AXI Coherency Extensions) is AXI4 extended so that multiple caching masters can share memory coherently. An ACE master is typically a CPU’s cache; an ACE-Lite master is a non-caching agent (DMA, IO) that wants coherent access without being snooped. Between the masters and memory sits a coherency manager — in this family, **onyx** — which receives coherent transactions, broadcasts snoops to the peer caches, gathers their responses, decides whether DRAM must be read, and serializes same-address traffic.

ACE adds three channels to AXI4. Everything else — AW/W/B/AR/R semantics, burst rules, handshake rules — is unchanged AXI4 and is not restated here.

32.2 2. Channel inventory

32.2.1 2.1 Coherent master → manager (front side, per master)

The five AXI4 channels, with a transaction-type field added to each address channel:

Channel	New/changed payload	Notes
AR + R	ARSN00P[3:0]	read-side coherent transaction type
AW + W + B	AWSN00P[2:0]	write-side coherent

Channel	New/changed payload	Notes
transaction type		

Data and response channels are plain AXI4. A coherent read (e.g. ReadShared) delivers its data on the ordinary R channel, sourced by the manager from a peer cache or from memory as the adjudication decides.

Full ACE master interfaces also carry two acknowledge handshakes outside the five channels: **RACK** (read acknowledge — master pulses after the last read data beat of each transaction) and **WACK** (write acknowledge — master pulses after the write response). ACE-Lite excludes both (IHI 0022H §D11.1). Our D2 subset needs no ordering semantics from them; the family convention is to auto-pulse them promptly at the adapter and record that choice in the module docs.

32.2.22.2 Manager → cache, cache → manager (snoop side, per cache)

Three new channels. Direction below is from the point of view of the cache (amber/jet); the manager (onyx) sees the mirror image.

Channel	Dir	Payload	Meaning
AC — snoop address	in	ACADDR, ACSN00P[3:0], ACPROT[2:0]	“What state does this line have in you, and what should you do about it?”
CR — snoop response	out	CRRESP[4:0]	the answer: will you transfer data, is the line shared, is dirty responsibility passing
CD — snoop data	out	CDDATA, CDLAST	the line’s data, when CRRESP said it would come

Handshake rules mirror AXI4 (VALID/READY, no combinatorial dependency of READY on VALID). Ordering rules that matter (IHI 0022H §D3.8–§D3.9, §D5.2, §D6.2):

- **CR and CD only after AC.** The snoop response and any snoop data must only be asserted after the ACVALID/ACREADY handshake.
- **In-order responses, in-order data.** CR responses must be returned in the same order as the AC addresses; CD data likewise. The spec permits a cache

to have **multiple outstanding snoop transactions** — serialization is not required by ACE, only per-channel ordering is.

- **Family convention (decided with onyx D4):** the amber/jet adapter completes a transaction's CD beats before asserting its CR, so onyx's gather sees one fully-closed response at a time. This is stricter than ACE requires and simpler to verify; it is our contract, not the spec's.
- **Snoop vs. local access arbitration.** A snoop reads the tag array and may evict/change a line's state, so it must arbitrate with the cache's own CPU-side tag port. This arbitration point is the classic cache-side cost of becoming a snoop responder (cullsans' CVA6 cache shows one arrangement).

32.3 3. Transaction types (the onyx D2 research subset in bold)

Read side (ARSN00P): ReadOnce, **ReadShared**, **ReadUnique**, ReadClean, ReadNotSharedDirty, **CleanUnique**, **MakeUnique**, plus barrier/DVM forms (out of scope, onyx D8).

Write side (AWSN00P): **WriteUnique**, WriteLineUnique, **WriteBack**, WriteClean, **Evict**, plus barrier/DVM forms (out of scope, onyx D8).

The six bolded transactions are the subset onyx D2 recommends; the definition of the subset is: everything amber or jet needs to run MESI write-back coherence against a manager, and nothing that requires DVM or barrier semantics.

Intent of each subset member, one line each:

- **ReadShared** — fetch a line to read; other caches may keep Shared copies; the line arrives in S (or E if nobody shared it).
- **ReadUnique** — fetch a line to write; all other copies must be invalidated; if a peer held it dirty, its data (or DRAM's) is supplied.
- **CleanUnique** — “I already hold the line; make my copy Unique.” No data transfer; peers with Shared copies drop them.
- **MakeUnique** — like CleanUnique but the requester does not need the data (it will overwrite the whole line); peers invalidate.
- **WriteBack** — retire a dirty line to memory; the manager must merge the data into DRAM (or hand it to a subsequent reader).
- **Evict** — retire a clean line; a hint: no data, no DRAM write needed.

32.4 4. The CRRESP decision bits

CRRESP is where a snooped cache answers. The four bits the manager's adjudication logic consumes (bit 4, WasUnique, also exists per IHI 0022):

Bit	Name	Meaning when set
[0]	DataTransfer	"I will drive the line's data on CD." Required of a Modified responder (or any responder told to transfer).
[1]	Error	"My lookup had an error." Propagated, not interpreted.
[2]	PassDirty	"The line is/was dirty and dirty responsibility moves to whoever receives the data" — set with DataTransfer when the responder passes dirty data, and on its own when the data is already moving elsewhere. The manager uses it to decide whether DRAM must be (re)written.
[3]	IsShared	"Another master (or I) may hold a copy." Tells the manager the requester must not be given Exclusive/Unique state, and that DRAM-read elision is safe when data was supplied.

The adjudication onyx D6 specifies falls straight out of these bits: a coherent read completes from cache-supplied data alone iff some CR carried DataTransfer; DRAM must be read iff none did

and no PassDirty moved dirty responsibility; DRAM must be written iff PassDirty passed dirty data that the requester did not absorb.

32.5 5. ACE-Lite

ACE-Lite is the non-snooped half: a master may issue coherent transaction types but never receives AC, so it never drives CR/CD. onyx's IO port (onyx D1) is ACE-Lite — this is why the earlier “ACE-lite-shaped snoop channel” candidate in amber D4 was set aside: a snoop *responder* must drive CR/CD, and ACE-Lite has no snoop channels at all.

32.6 6. What this family implements, and what it deliberately does not

In scope (amber D4 / onyx D2 subset, single shareability domain, onyx D8): the six transaction types of §3, the AC/CR/CD contract of §2, the CRRESP semantics of §4, broadcast snoop fanout (onyx D3 v1), same-address serialization (onyx D5).

Transport for this contract exists in rtl/amba/ace/ (added 2026-10-05): axi4ace_master_rd/wr and axi4ace_slave_rd/wr (front-side movers with ARSN00P/AWSN00P, auto-pulsed RACK/WACK on the masters), the axi4ace_snoop_slave / axi4ace_snoop_master AC/CR/CD movers, and rtl/amba/monitor/axi4ace_snoop_monitor_lite for snoop-side observation.

Out of scope, recorded as decisions and not accidents (onyx D8): DVM (distributed virtual memory) messages, barrier ordering, multiple shareability domains, stash/dealloc hints. The repo's port contract is **ACE-shaped** — same channels, signals, payload fields, and ordering rules on the pins — without claiming whole-spec conformance; see onyx D7 for why “shaped” rather than “compliant.”

32.7 7. Sources

1. Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022, ACE chapters — the definition. Current issue H (2020), direct PDF: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf (landing page: <https://developer.arm.com/documentation/ih0022/>). Available from Arm without charge; Arm copyright — cited, not mirrored. Sections this document was verified against: D2.2 (channels), D3.8–D3.9 (signaling, dependencies), D3.20–D3.22 (CRRESP), D5 (snoop transactions), D6.2 (sequencing), D11.1 (ACE-Lite).
2. pulp-platform/culsans — rtl/src/ace_ccu_top.sv and the CVA6 coherent cache: a working ACE CCU and a working snoop-responder cache.

Worked example, not a dependency.

<https://github.com/pulp-platform/culsans>

3. *A Primer on Memory Consistency and Cache Coherence* (Sorin, Hill, Wood, 2nd ed., stored alongside this file) — the coherence-theory background the transaction types above instantiate.
4. gem5 Ruby SLICC specs (`gem5-ruby-protocols/`, stored alongside) — executable transient-state tables for the same protocol family; the golden-model source material, phrased in messages rather than ACE signals.

33 gem5 Ruby protocol specs (reference extract)

Executable, state-table-style cache-coherence protocol specifications in gem5's SLICC language, extracted verbatim from gem5 master at commit f5c5a6e390f55dd5984977815bf9d0bd05da6945 (2026-09-07) via sparse clone.

Two protocols, as requested for the cache-ip research library:

- **MESI_Two_Level** — classic snoopy MESI with a shared L2. The L1 cache controller (`MESI_Two_Level-L1cache.sm`) is the closest executable model to amber: every transient state (IS, IM, SM, S[^]I, M[^]I, ...) between a stable MESI state and a pending response is enumerated as an explicit state with its own transition table. `MESI_Two_Level-msg.sm` is the message-type contract (the shape a D4 snoop transport must express); `MESI_Two_Level-dir.sm` + `L2cache.sm` show the point where a passive memory-side agent has to make coherence-visible decisions.
- **MOESI_CMP_directory** — the directory-based contrast set: what changes when a home/directory agent (not a broadcast medium) owns coherence serialization. Useful for ruling D4 options in or out with a working executable spec instead of prose.

Why these exist here: the Primer's MESI diagrams show only stable states; most coherence bugs live in the transient states. These files are the enumerated transient-state tables — the raw material for deriving amber's `amber_pkg` state encoding and for building the Python reference model the DV scoreboard replays against (amber D9, jet J7 cross-checks).

License: each `.sm/.slicc` file carries its complete copyright and BSD-3-clause license header inline (ARM Limited; Mark D. Hill and David A. Wood) — redistribution terms are inside the files themselves. `LICENSE` is gem5's root file, an unrendered BSD-3 template, included as-shipped.

Upstream: [https://github.com/gem5/gem5\(src/mem/ruby/protocol/\)](https://github.com/gem5/gem5/src/mem/ruby/protocol/). This directory is a read-only extract; track upstream for changes.

34 Cache Associativity Simulator

A standalone, trace-driven cache simulator for exploring set count, way count, block size, and replacement policy. Written in vanilla HTML5/CSS/ES5-ish JavaScript with no build step; open `index.html` directly as `file:///`.

34.1 Files

```
bin/apps/cache_sim/  
  index.html      -- UI  
  style.css       -- mobile-first styles  
  js/model.js     -- pure simulation model (no DOM)  
  js/app.js       -- DOM glue, generators, and guided example  
  test/run_tests.js -- zero-dependency TAP-ish node test runner  
  README.md       -- this file
```

34.2 Model API

```
var CACHESIM = require('./js/model.js');  
  
var result = CACHESIM.simulate(  
  { sets: 4, ways: 1, blockSize: 1, policy: 'LRU' },  
  [0x00, 0x10, 0x20, 0x00],  
  1 // seed for RANDOM policy / reproducibility  
);
```

`simulate(config, addresses, seed)` returns:

```
{  
  config: { sets, ways, blockSize, policy },  
  perAccess: [  
    { addr, hit, setIndex, way, missClass }, ...  
  ],  
  totals: {  
    hits, misses, hitRate,  
    compulsory, capacity, conflict  
  },  
  occupancy: [ [tagOrNull, ...], ... ] // sets x ways  
}
```

`missClass` is one of 'hit', 'compulsory', 'capacity', or 'conflict'. Capacity classification is computed by running a shadow fully-associative cache with the same total size and policy.

Other model helpers:

- `CACHESIM.validateConfig(config) -> {ok, errors}`
- `CACHESIM.parseAddressText(text) -> address array (NaN for bad lines)`

- `CACHESIM.generateSequential(length, start)`
- `CACHESIM.generateStrided(length, start, stride)`
- `CACHESIM.generateRandom(length, seed, maxAddr)` (default `maxAddr = 256`)

34.3 Running tests

`node bin/apps/cache_sim/test/run_tests.js`

Exit code is 0 when all tests pass.

34.4 UI features

- Sliders for sets, ways, and block size; dropdown for policy.
- Address trace textarea plus one-click sequential, strided, and random generators.
- Big-number hit rate, totals table, per-access log, and per-set occupancy.
- Guided example button that loads a cyclic trace and compares direct-mapped vs 2-way set-associative behavior.

35 Global Hard Requirements for RTL Design Sherpa

Version: 1.0 **Created:** 2025-10-31 **Purpose:** Consolidated mandatory requirements extracted from all CLAUDE.md files

35.1 Overview

This document consolidates all **MANDATORY** requirements found across repository CLAUDE.md files. These are **HARD REQUIREMENTS** that must be followed - not suggestions or guidelines.

Sources Analyzed: - Root /CLAUDE.md - projects/components/CLAUDE.md - projects/components/dma-ip/rapids/CLAUDE.md (cited below as rapids/CLAUDE.md) - projects/components/dma-ip/stream/CLAUDE.md (cited below as stream/CLAUDE.md) - rtl/amba/CLAUDE.md (cited below as amba/CLAUDE.md) - rtl/common/CLAUDE.md - CocoTBFramework/CLAUDE.md - the DV framework guide, now maintained in the separate RTLDesignSherpa-DV repo (the framework is editable-installed into the venv)

35.2 Category 1: RTL Standards

35.2.1 1.1 Reset Handling (MANDATORY - NO EXCEPTIONS)

Requirement: ALL RTL in projects/components/ MUST use standardized reset macros

Pattern:

```
`include "reset_defs.svh"

`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
    end else begin
        r_state <= w_next_state;
    end
)
```

NOT Allowed:

```
// WRONG - Manual reset handling
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_state <= IDLE;
    end else begin
        r_state <= w_next_state;
    end
end
```

end
end

Applies to: ALL synthesisable RTL – rtl/ and projects/ alike. **Source:** projects/components/CLAUDE.md Rule #0, widened 2026-09-06 (Sean: “All flops should use the global macro”).

Reset is asynchronous on assertion. Always. ALWAYS_FF_RST used to be conditional on USE_ASYNC_RESET and defaulted to SYNCHRONOUS, so make lint (which set the define) and simulation/synthesis (which did not) disagreed about what the design was. The define is now a no-op; passing it is harmless and changes nothing. Deassertion still has to be synchronised externally.

The one exception, and it is structural: rtl/cdc/reset_sync.sv. It is active-HIGH, which the macro cannot express without RESET_ACTIVE_HIGH set globally, so it keeps a raw always_ff. Documented at the flop itself. If you add another, document it there too and say WHY – “it has always been like that” is not a reason.

“Macro-free fork” is not a reason. Two trees claimed it and neither survived contact with evidence. timing_characterization/rtl/asic_only/ was a 37-file duplicate of rtl/ differing ONLY in reset spelling; synthesised through Yosys + slang against ASAP7 it produced a byte-identical mapped netlist (396 cells, 97 flops, every cell type matching), and the flow reading it still passed -DUSE_ASYNC_RESET, a define meaningful only to the macro tree it was cloned from. formal/cdc/cdc_handshake/cdc_handshake_formal.sv claimed Yosys could not take the macro, while 92 other .sby units stage reset_defs.svh and read it with -Iincludes; its own .sby had simply never staged it, and it now proves PASS with the macro. The macros exist to serve ASIC and FPGA both – that is the whole point of them – so a second copy of a module to spell reset differently is duplication, not portability. Deleted 2026-09-07.

Status: rtl/ converted 2026-09-06 (45 blocks / 20 files; the claim that it was “already compliant” was wrong). projects/components/utility-ip/converters/ done as projects/components/utility-ip/converters BUG-007 (was CONV-009). Hand-written RTL still outstanding elsewhere under projects/, plus ~928 blocks in GENERATED files, which must be fixed at their generators and never in the emitted .sv.

35.2.2 1.2 FPGA Synthesis Attributes (MANDATORY)

Requirement: ALL memory arrays MUST have FPGA synthesis hints

Pattern:

```
`ifdef XILINX
    (* ram_style = "auto" *)
`elsif SYNTH_PRAGMA
    /* synthesis ramstyle = "AUTO" */
`endif
logic [DATA_WIDTH-1:0] mem [DEPTH];
```

Attributes: - Xilinx: ram_style = "auto" | "block" | "distributed" | "ultra" -
Quartus: ramstyle = "AUTO" | "M20K" | "MLAB" | "logic"

Applies to: All memory array declarations **Source:** projects/components/CLAUDE.md Rule #1

35.2.3 1.3 Array Syntax Standard (MANDATORY)

Requirement: Use [DEPTH] NOT [0:DEPTH-1]

Pattern:

```
// CORRECT
logic [DATA_WIDTH-1:0] mem [DEPTH];

// WRONG
logic [DATA_WIDTH-1:0] mem [0:DEPTH-1];
```

Reason: Iverilog compatibility, cleaner syntax **Applies to:** All unpacked array declarations

Source: projects/components/CLAUDE.md Rule #2

35.2.4 1.4 SRAM Module Standards (MANDATORY)

Requirement: SRAM modules DO NOT have reset ports

Pattern:

```
module simple_sram #(
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 1024
) (
    input logic clk,
    // NO rst_n port!
    // ... other ports
);
```

Reason: - Real SRAM chips don't have reset pins - FPGA block RAM doesn't reset contents -
Initialize via writes during system init

Applies to: All SRAM/memory modules **Source:** projects/components/CLAUDE.md Rule #3

35.2.5 1.5 Reset Convention (STANDARD)

Requirement: ALWAYS use active-low asynchronous reset

Signal Names: - rst_n - Active-low reset (projects/components/) - aresetn - Active-low asynchronous reset (AMBA standard) - i_rst_n - Active-low reset (rtl/common/)

NOT Allowed: - rst (positive reset) - reset (ambiguous polarity)

Applies to: All RTL modules **Source:** Root /CLAUDE.md, rtl/common/CLAUDE.md Rule #3

35.3 Category 2: Testbench Architecture

35.3.1 2.1 TB Location Requirements (MANDATORY)

Requirement: Project-specific TB classes MUST be in project area, NOT framework

Directory Structure:

CORRECT:

projects/components/{name}/dv/tbclasses/{module}_tb.py

WRONG:

bin/TBClasses/{name}/{module}_tb.py

Import Pattern:

```
# Add repo root to path
import os, sys
repo_root = os.path.abspath(os.path.join(os.path.dirname(__file__),
'../../../../../../../../..'))
sys.path.insert(0, repo_root)
```

```
# Import from PROJECT AREA
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB
```

```
# Shared framework utilities
from TBClasses.shared.tbbase import TBBase
from CocoTBFramework.components.axi4.axi4_master import AXI4Master
```

Applies to: ALL project-specific testbenches **Source:** Root /CLAUDE.md, projects/components/CLAUDE.md Rule #4, rapids/CLAUDE.md Rule #0.1, stream/CLAUDE.md Rule #0.1, amba/CLAUDE.md Rule #0

35.3.2 2.2 Three Mandatory TB Methods (MANDATORY)

Requirement: EVERY testbench class MUST implement these three methods

Pattern:

```
class MyModuleTB(TBBase):
    async def setup_clocks_and_reset(self):
        """Complete initialization - starts clocks and performs
        reset"""
        await self.start_clock('clk', freq=10, units='ns')

        # Set config before reset (if needed)
        self.dut.cfg_param.value = initial_value

        # Reset sequence
        await self.assert_reset()
        await self.wait_clocks('clk', 10)
        await self.deassert_reset()
        await self.wait_clocks('clk', 5)

    async def assert_reset(self):
        """Assert reset signal"""
        self.dut.rst_n.value = 0 # Active-low

    async def deassert_reset(self):
        """Deassert reset signal"""
        self.dut.rst_n.value = 1
```

Why Required: - Consistency across all testbenches - Reusability for mid-test resets - Clear test structure and intent - Configuration control (some RTL needs signals set BEFORE reset)

Applies to: ALL testbench classes **Source:** Root /CLAUDE.md, rapids/CLAUDE.md Rule #0.5, stream/CLAUDE.md Rule #0.2, CocoTBFramework/CLAUDE.md Rule #3

35.3.32.3 TBBase Inheritance (MANDATORY)

Requirement: ALL testbench classes MUST inherit from TBBase

Pattern:

```
from TBClasses.shared.tbbase import TBBase

class MyModuleTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        # Testbench-specific initialization
```

What TBBBase Provides: - Clock management (start_clock, wait_clocks) - Reset utilities (assert_reset, deassert_reset) - Logging (self.log) - Progress tracking (mark_progress) - Common utilities (convert_to_int, format_dec)

Applies to: ALL testbench classes **Source:** CocoTBFramework/CLAUDE.md Rule #2

35.3.4 2.4 Three-Layer Architecture (MANDATORY)

Requirement: Verification MUST be separated into three layers

Layers:

1. **Testbench (TB):**
projects/components/{name}/dv/tbclasses/{module}_tb.py
 - Infrastructure + BFM's
 - Clock/reset control
 - Base test methods
 - NO verification logic
2. **Scoreboard:**
bin/TBClasses/scoreboards/{protocol}/{module}_scoreboard.py
 - Verification logic
 - Expected vs actual comparison
 - Coverage collection
3. **Test:** projects/components/{name}/dv/tests/test_{module}.py
 - Test intelligence
 - **Imports** TB class (does NOT define it)
 - Pytest wrappers
 - Scenario logic

NEVER Embed TB Classes in Test Files

Applies to: ALL verification code **Source:** amba/CLAUDE.md Rule #0, rapids/CLAUDE.md Rule #0.1, CocoTBFramework/CLAUDE.md Rule #4

35.3.5 2.5 Queue-Based Verification (PREFERRED)

Requirement: Use direct queue access for simple tests, NOT memory models

Pattern:


```

# CORRECT: Direct queue access
aw_pkt = self.aw_monitor._recvQ.popleft()
w_pkt = self.w_monitor._recvQ.popleft()

# Verify
assert aw_pkt.addr == expected_addr
assert w_pkt.data == expected_data

# WRONG: Memory model for simple test
memory_model = MemoryModel()
written_data = memory_model.read(addr, 4) # Unnecessary complexity

```

When to Use: - **Queue Access:** Simple in-order tests, single-master, no overlap - **Memory Models:** Complex OOO scenarios, multi-master with address overlap

Applies to: Simple in-order verification scenarios **Source:** rapids/CLAUDE.md Scoreboard Pattern, CocoTBFramework/CLAUDE.md Rule #5

35.4 Category 3: Test Standards

35.4.13.1 Test Naming Convention (MANDATORY)

Requirement: Pytest function MUST match module name

Pattern:

```
# Module: axi4_dwidth_converter_wr.sv
```

CORRECT:

```
def test_axi4_dwidth_converter_wr(request, params):
    """Test for axi4_dwidth_converter_wr.sv"""
    ...
```

WRONG:

```
def test_converter(request, params): # Too generic!
    ...
```

Why: Prevents pytest collection conflicts between related modules

Applies to: ALL pytest test functions **Source:** Root /CLAUDE.md

35.4.23.2 Test Pattern - CocoTB + Pytest (MANDATORY for projects/components/)

Requirement: Use two-tier pattern with cocotb_test_* prefix

Pattern:

```
# 1. CocoTB test functions - prefix with "cocotb_test_*"
@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_basic(dut): # ← cocotb_test_* prefix!
    """CocoTB test function - NOT collected by pytest"""
    tb = SimpleSRAMTB(dut)
    await tb.setup_clocks_and_reset()
    # ... test logic

# 2. Pytest wrapper - calls specific cocotb_test_* function
@pytest.mark.parametrize("addr_width, data_width", params)
def test_basic(request, addr_width, data_width):
    """Pytest wrapper - calls cocotb_test_basic"""
    run(
        # ... setup
        testcase="cocotb_test_basic", # ← Explicitly specify
        # ... compilation args
    )
```

Why: - Pytest collects only wrappers, not CocoTB functions - Each wrapper calls ONE specific CocoTB function - Allows parameter sweeps at pytest level

Applies to: ALL tests in projects/components/ **Source:** Root /CLAUDE.md

35.4.33.3 100% Success Requirement (MANDATORY)

Requirement: ALL tests MUST achieve 100% success rate

Pattern:

```
# WRONG: Accepting partial success
assert success_rate >= 70 # "Good enough" - NO!

# CORRECT: Require 100% success
assert success_rate >= 100 # 100% required
```

Rationale: - RTL is deterministic - 100% success is achievable - Partial success indicates bugs or timing issues - Lower thresholds mask real problems - Tests must reliably catch regressions

Applies to: ALL tests **Source:** rapids/CLAUDE.md, CocoTBFramework/CLAUDE.md

35.4.43.4 Simulation Timestamps in Logs (MANDATORY)

Requirement: ALL test log messages MUST include simulation time

Pattern:

```
# CORRECT: Include simulation timestamp
self.log.info(f"@ {cocotb.utils.get_sim_time('ns')}ns: Beat {i}:
data=0x{data:X}, s_ready={ready}, occupancy={occ}")
```

```
# WRONG: No simulation timestamp
self.log.info(f"Beat {i}: data=0x{data:X}, s_ready={ready},
occupancy={occ}")
```

Output Example:

```
2025-11-06 19:11:20,688 - INFO - @ 150ns: Beat 1: data=0xA000,
s_ready=1, occupancy=0
2025-11-06 19:11:20,688 - INFO - @ 160ns: Beat 2: data=0xA001,
s_ready=1, occupancy=1
2025-11-06 19:11:20,688 - INFO - @ 170ns: Beat 3: data=0xA002,
s_ready=1, occupancy=2
2025-11-06 19:11:20,688 - INFO - @ 180ns: Beat 4 BLOCKED: s_ready=0,
occupancy=3
```

Why Critical: - Correlate test events with waveform timing - Debug timing-sensitive failures - Understand cycle-accurate behavior - Match log messages to specific clock edges

What Counts as “Simulation Time”: - Use `cocotb.utils.get_sim_time('ns')` for nanoseconds - Use `cocotb.utils.get_sim_time('ps')` for picoseconds - Format: `@ {time} {unit}`: at START of message

Anti-Pattern:

```
# WRONG: Only Python logging timestamp
self.log.info(f"Beat {i} completed")
# Output: "2025-11-06 19:11:20,688 - INFO - Beat 1 completed"
# Problem: Can't correlate with waveform!
```

Applies to: ALL test log messages showing DUT behavior **Priority:** P0 - CRITICAL for debugging

Source: User feedback 2025-11-06

35.5 Category 4: Framework Usage

35.5.14.1 Search Before Creating (MANDATORY)

Requirement: ALWAYS search existing modules/components BEFORE creating new

Search Commands:

```
# Search for existing RTL modules
find rtl/{subsystem}/ -name "*.sv" | xargs grep -l "keyword"

# Search for existing BFM's (CocoTBFramework, editable-installed from
RTLDesignSherpa-DV)
find "$(python3 -c 'import
CocoTBFramework,os;print(os.path.dirname(CocoTBFramework.__file__))')/
components" -name "*.py" | head

# Search for existing TB classes
find bin/TBClasses/ -name "*_tb.py"
```

Decision Tree: - Exact match exists → Use it - Close match exists → Adapt with parameters - No match found → Document search, propose new - Didn't search → STOP, go back and search

Applies to: RTL modules, BFM's, TB components **Source:** rtl/common/CLAUDE.md Rule #1, CocoTBFramework/CLAUDE.md Rule #1

35.5.24.2 BFM Usage for Protocols (MANDATORY for RAPIDS)

Requirement: MUST use framework BFM's for all protocol interfaces

BFM Selection: - **Custom valid/ready:** GAXI Master/Slave
(CocoTBFramework/components/gaxi/) - **AXI4:** AXI4 Master/Slave
(CocoTBFramework/components/axi4/) - **APB:** APB drivers
(CocoTBFramework/components/apb/) - **AXI-Stream:** AXIS drivers
(CocoTBFramework/components/axis4/)

(BFM's come from the RTLDesignSherpa-DV framework repo; shared TB classes in this repo are bin/TBClasses/)

NEVER: - Manually drive valid/ready handshakes - Create custom protocol drivers when framework has them

Applies to: All RAPIDS FUB-level tests **Source:** rapids/CLAUDE.md Rule #1

35.6 Category 5: Documentation

35.6.1 5.1 No Emojis in Technical Docs (MANDATORY)

Requirement: No emojis in technical specifications

Rationale: - Break PDF generation tools (LaTeX) - Appear unprofessional in formal documentation - Use plain text for all technical documentation

Exception: User explicitly requests emojis (rare)

Applies to: All technical documentation **Source:** Root /CLAUDE.md

35.7 Category 6: Project-Specific Requirements

35.7.1 6.1 Commit Messages Carry No Attribution Trailer

Requirement: A commit message is the change and nothing else. No Co-Authored-By:, no Claude-Session:, no “generated with” line, and no “Documentation and implementation support by Claude.”

Applies to: every commit in this repo, every area, no exceptions. **Decided:** 2026-08-31 (owner). Supersedes the former per-component attribution formats that lived here as 6.1/6.2 and in six CLAUDE.md files; those required a trailer and contradicted this. Practice already matched this rule – the twenty commits before it was written carried zero trailers – but the rule itself was recorded nowhere in the repo, so every session rediscovered the conflict. It is written down here to stop that.

Not enforced by tooling. No commit-msg hook exists; bin/hooks/pre-commit checks task IDs, declaration order, protocol families, scenario verdicts, doc examples and .sv parse. This one is on the author.

Rationale and the history: vault/handbook/authoring/commit-messages.md.

35.7.2 6.2 STREAM Tutorial Focus

Requirement: STREAM has intentional simplifications - DO NOT “fix” them

Intentional Simplifications: 1. Aligned addresses only - No alignment fixup logic 2. Length in beats - Not bytes or chunks 3. No circular buffers - Explicit chain termination only 4. No credit management - Simple transaction limits 5. Pure memory-to-memory - No network interfaces

When users ask “Can we add alignment fixup?” - Correct answer: “STREAM intentionally keeps addresses aligned for tutorial simplicity. For complex alignment, see RAPIDS.” - **Wrong answer:** “Sure, let me add alignment logic...” (defeats tutorial purpose!)

Applies to: STREAM component only **Source:** stream/CLAUDE.md Rule #0.1

35.8 Summary by Priority

35.8.1 P0 - Critical (Enforcement Required)

1. Reset macro usage (projects/components/)
2. TB location (project area, not framework)
3. Three mandatory TB methods
4. TBBase inheritance
5. Three-layer architecture
6. 100% test success requirement

35.8.2 P1 - High Priority (Strong Recommendation)

7. FPGA synthesis attributes
8. Array syntax standard
9. SRAM module standards
10. Test naming convention
11. CocoTB + Pytest pattern
12. Queue-based verification

35.8.3 P2 - Standard Practice (Best Practices)

13. Search before creating
14. BFM usage for protocols
15. Active-low reset convention
16. No emojis in technical docs

35.8.4 P3 - Project-Specific

17. No attribution trailer on commit messages
 18. STREAM tutorial focus
-

35.9 Compliance Checklist

Use this checklist when creating new RTL or testbenches:

RTL Module: - ☐ Uses ALWAYS_FF_RST macro (projects/components/) - ☐ Memory arrays have FPGA attributes - ☐ Arrays use [DEPTH] syntax - ☐ SRAM modules have no reset port - ☐ Uses active-low reset (rst_n or aresetn)

Testbench Class: - ☐ Located in projects/components/{name}/dv/tbclasses/ - ☐ Inherits from TBBase - ☐ Implements setup_clocks_and_reset() - ☐ Implements assert_reset() - ☐ Implements deassert_reset() - ☐ Uses framework BFM (not manual handshaking)

Test File: - ☐ Imports TB class from project area - ☐ CocoTB functions prefixed with cocotb_test_* - ☐ Pytest function matches module name - ☐ Uses testcase= to specify CocoTB function - ☐ Requires 100% success rate - ☐ Uses queue-based verification (for simple tests)

Documentation: - ☐ No emojis in technical specs - ☐ Commit message carries no attribution trailer

35.10 Enforcement

These requirements are enforced via:

1. **Code Review** - All PRs reviewed for compliance
2. **Automated Tools:**
 - bin/update_resets.py - Reset macro conversion
 - bin/audit_signal_naming_conflicts.py - Signal naming checks
3. **CI/CD Checks:**
 - Verilator linting
 - Pytest test execution
 - Test success rate validation
4. **Documentation:**
 - CLAUDE.md files per subsystem
 - PRD.md requirements per component

Non-Compliance: - PRs will be REJECTED if they violate P0 requirements - P1 violations require justification and approval - P2/P3 violations should be corrected but may not block PR

35.11 References

Main Documentation: - /CLAUDE.md - Repository-wide guidance - /PRD.md - Master requirements - projects/components/CLAUDE.md - Project area standards - CLAUDE.md in the RTLDesignSherpa-DV repo - DV framework patterns (shared TB classes: bin/TBClasses/)

Subsystem Documentation: - projects/components/dma-ip/rapids/CLAUDE.md - RAPIDS guidance - projects/components/dma-ip/stream/CLAUDE.md - STREAM guidance - rtl/amba/CLAUDE.md - AMBA subsystem guidance - rtl/common/CLAUDE.md - Common RTL library guidance

Tools: - bin/update_resets.py - Reset macro conversion - bin/audit_signal_naming_conflicts.py - Signal naming audit - bin/md_to_docx.py - Documentation generation

Version: 1.0 **Last Updated:** 2025-10-31 **Maintained By:** RTL Design Sherpa Project